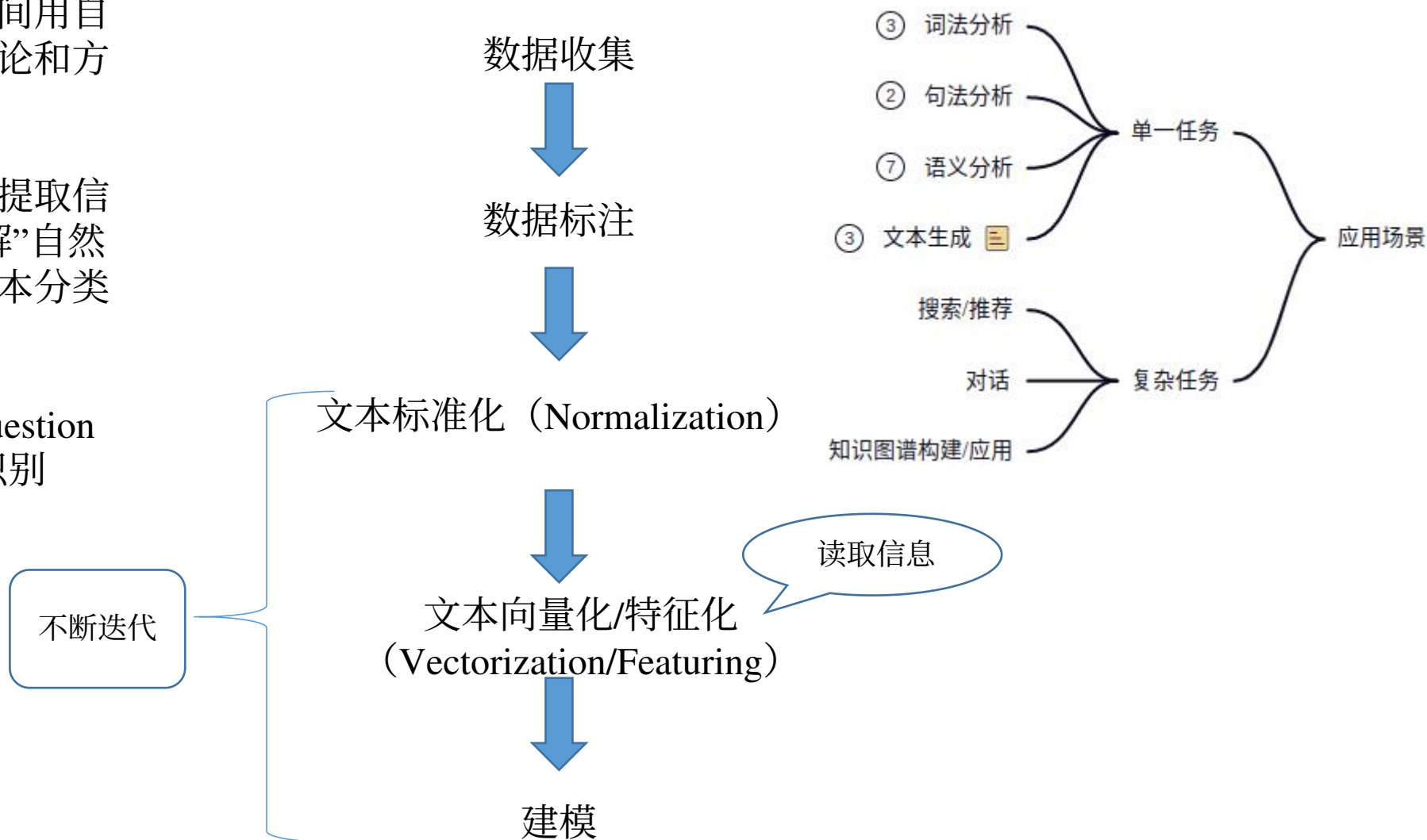
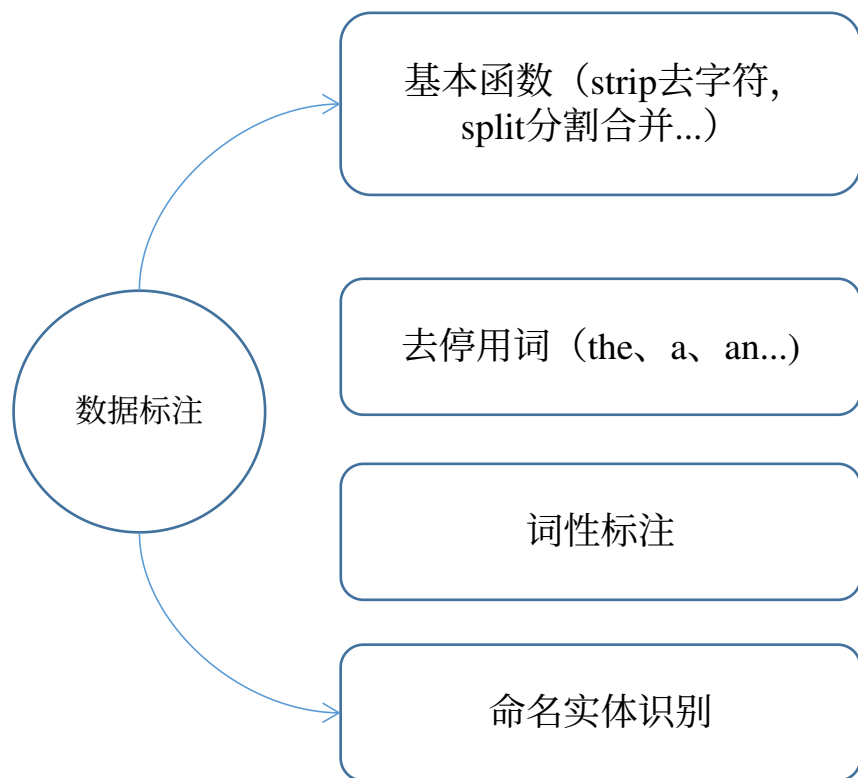


自然语言处理 (Natural Language Processing, NLP)

- 研究对象：人与计算机之间用自然语言进行有效通信的理论和方法。
- 研究目的：从文本数据中提取信息。让计算机处理或“理解”自然语言，执行自动翻译、文本分类和情感分析等。
- 技术范畴：问答系统 (Question Answering, QA)、语音识别 (Speech Recognition)





```
Today's weather is good , very windy and sunny.
['weather', '.', 'good', 'sunny', 'today', 'windy', ',']

(PERSON Edison/NNP)
went/VBD
to/TO
(ORGANIZATION Tsinghua/NNP University/NNP)
today/NN)
```

TF-IDF

IDF越大，说明词条越具有很好的类别区分能力。

TF(词频, Term Frequency):
表示词条(关键字)在文本中出现的频率。

IDF: 逆向文件频率(Inverse Document Frequency)

$$TF_w = \frac{\text{在某一类中词条}w\text{出现的次数}}{\text{该类中所有的词条数目}}$$

$$IDF = \log \frac{\text{语料库的文档总数}}{\text{包含词条}w\text{的文档数}+1} \quad (\text{分母之所以要加1, 是为了避免分母为0})$$

TF-IDF

某一特定文件内的高
词语频率

该词语在整个文件集
合中的低文件频率

高权重的TF-
IDF

过滤掉常见的词语，
保留重要的词语。

词向量

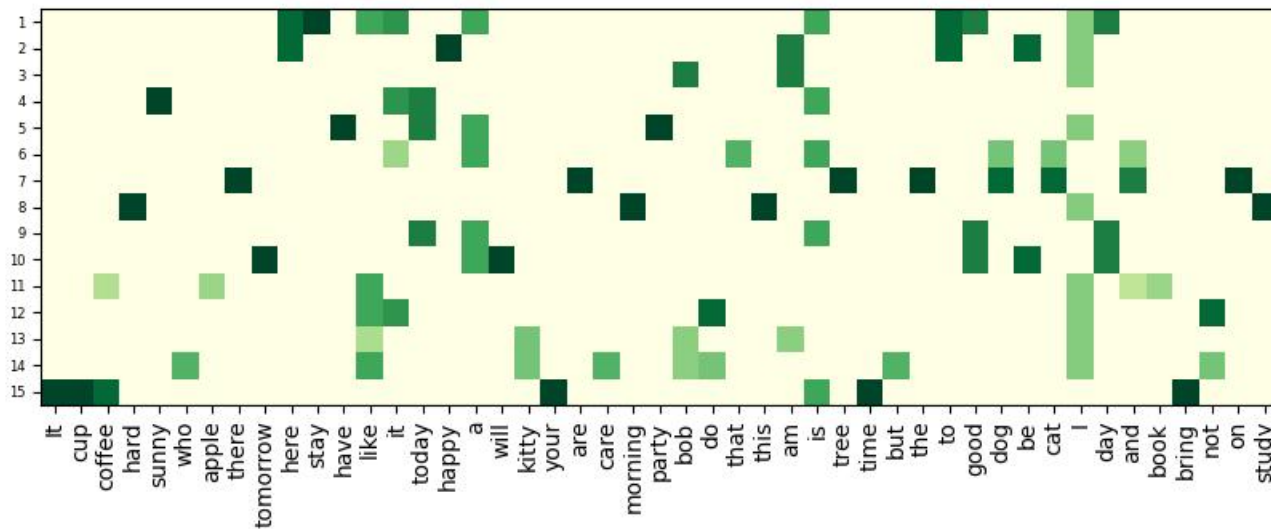
一篇文章是一个
向量

15 句话 47个不同词汇

```
docs = [  
    "it is a good day, I like to stay here",  
    "I am happy to be here",  
    "I am bob",  
    "it is sunny today",  
    "I have a party today",  
    "it is a dog and that is a cat",  
    "there are dog and cat on the tree",  
    "I study hard this morning",  
    "today is a good day",  
    "tomorrow will be a good day",  
    "I like coffee, I like book and I like apple",  
    "I do not like it",  
    "I am kitty, I like bob",  
    "I do not care who like bob, but I like kitty",  
    "It is coffee time, bring your cup",  
]
```

9、10两句

TF-IDF算法是通过统计学的方式来将文章中的词的重要程度，转化成向量来表示一篇文章。



Continuous Bag of Words (CBOW)

numbers

"5 2 4 8 6 2 3 6 4",
 "4 8 5 6 9 5 5 6",
 "1 1 5 2 3 3 8",
 "3 6 9 6 8 7 4 6 3",
 "8 9 9 6 1 4 3 4",
 "1 0 2 0 2 1 3 3 3 3",
 "9 3 3 0 1 4 7 8",
 "9 9 8 5 6 7 1 2 3 0 1 0",

alphabets, expecting that 9 is close to letters

字母, 期望9接近字母, 9 夹杂在字母中

"a t g q e h 9 u f",
 "e q y u o i p s",
 "q o 9 p l k j o k k o p",
 "h g y i u t t a e q",
 "i k d q r e 9 e a d",
 "o p d g 9 s a f g a",
 "i u y g h k l a s w",
 "o l u y a o g f s",
 "o p i u y g d a s j d l",
 "u k i l o 9 l j s",
 "y g i s h k j l f r f",
 "i o h n 9 9 d 9 f a 9",

数据集 (向量化)

[前2索引, 前1索引, 后1索引, 后2索引]	
本身索引	
{去重元素: 索引}	
{索引: 去重元素}	

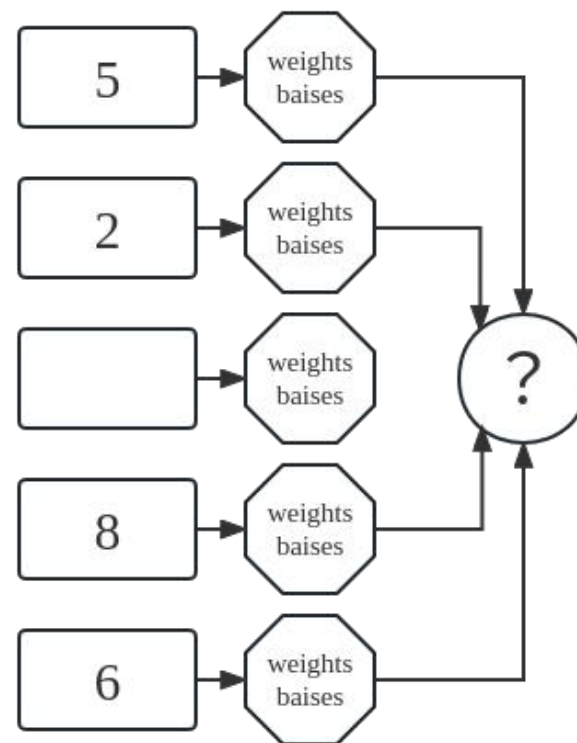
训练

挑一个要预测的词, 来学习这个词前后文中词语和预测词的关系。

目标

用前后文的词向量来预测句中的某个词

一起出现的向量在向量图上相距较近



$$(V_5 + V_2 + V_8 + V_6) / 4 = V_4$$

Embedding
(嵌入式、嵌入层)

one-hot (独热) 编码

小 普 喜 欢 星 海 湾 的 朋 友
0 1 2 3 4 5 6 7 8 9

小普喜欢星海湾的朋友	星海湾的朋友喜欢小普
[1 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 1 0]
[0 1 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0 1]
[0 0 1 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 1 0 0 0]
[0 0 0 1 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 1 0 0]
[0 0 0 0 1 0 0 0 0 0]	[1 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 1 0 0 0 0]	[0 1 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 1 0 0 0]	[0 0 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 1 0 0]	[0 0 0 1 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 1 0]	[0 0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 1]	[0 0 0 0 0 1 0 0 0 0 0]

我们的CBOW编码是将30维降为二维

此处本质是降维

计算简单，维度爆炸过于稀疏，占用资源

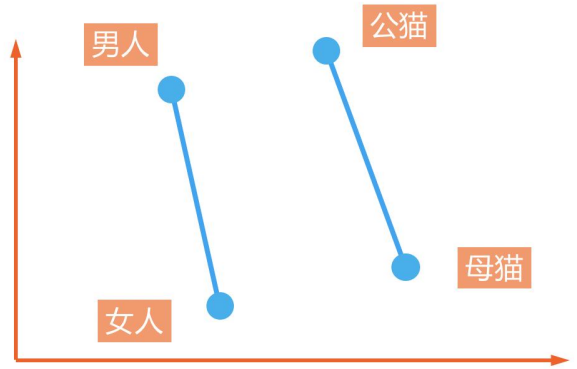
词向量都是彼此正交，体现不出相似关系



$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \\ w_{51} & w_{52} & w_{53} \\ w_{61} & w_{52} & w_{53} \end{bmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix}$$

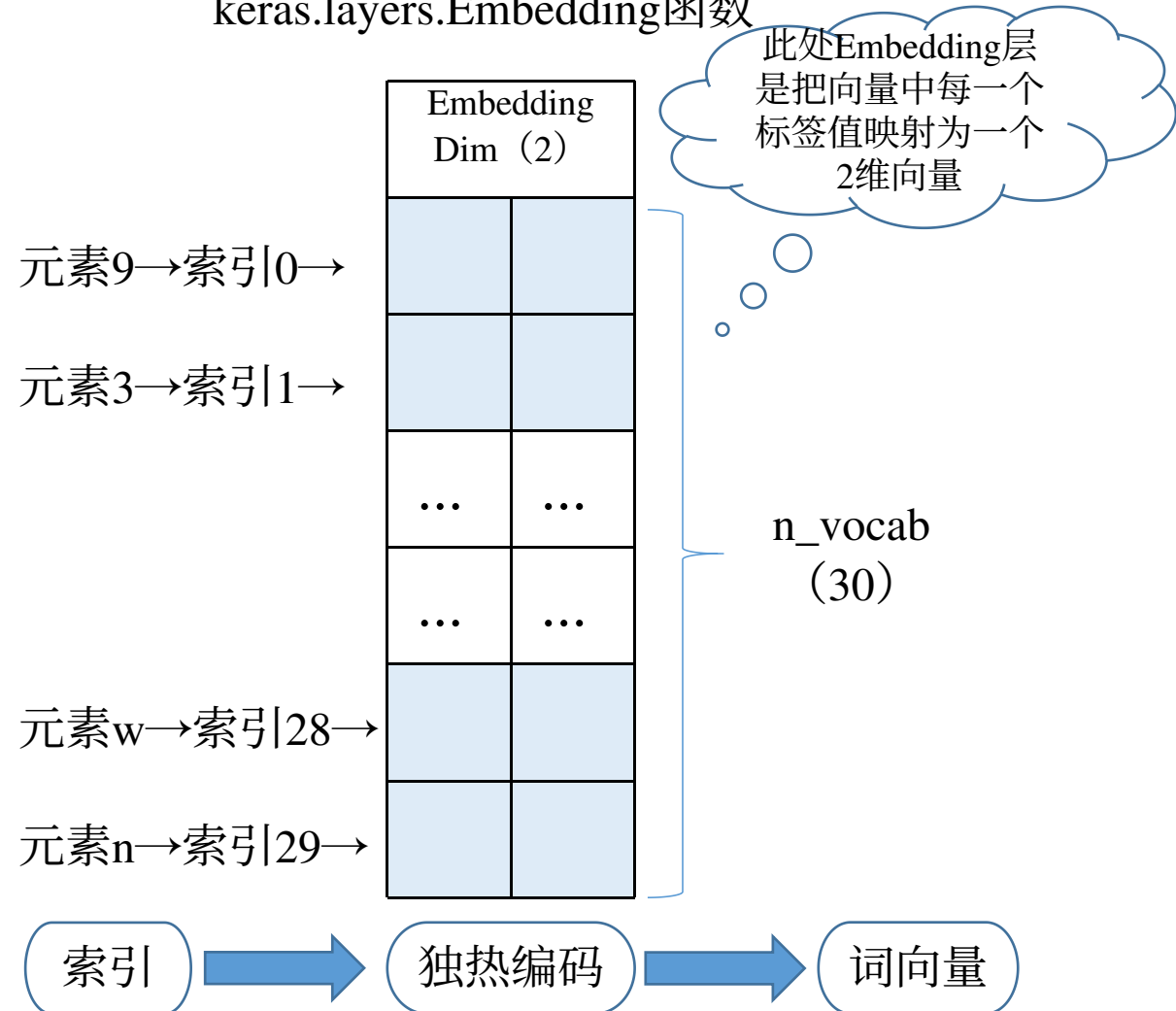
A * B = C



$$\begin{matrix} \text{男人} & - & \text{女人} \\ \approx & & \\ \text{公猫} & - & \text{母猫} \end{matrix}$$

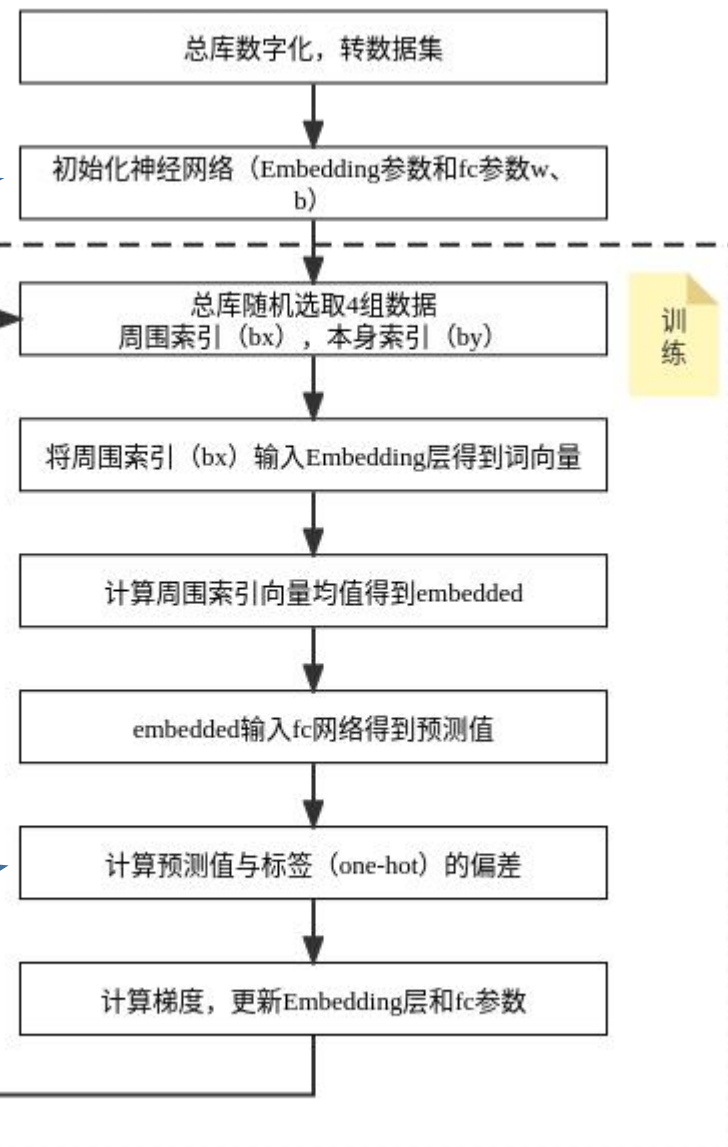
Embedding (嵌入式、嵌入层)

keras.layers.Embedding函数



fc输入为4X2矩阵
embedded, 输出为
4X30矩阵, 与每个
元素的预测概率
正相关

one-hot矩阵
相当于每个
元素概率列表



对比CBOW

目标：预测文本中某个字周围可能出现的词

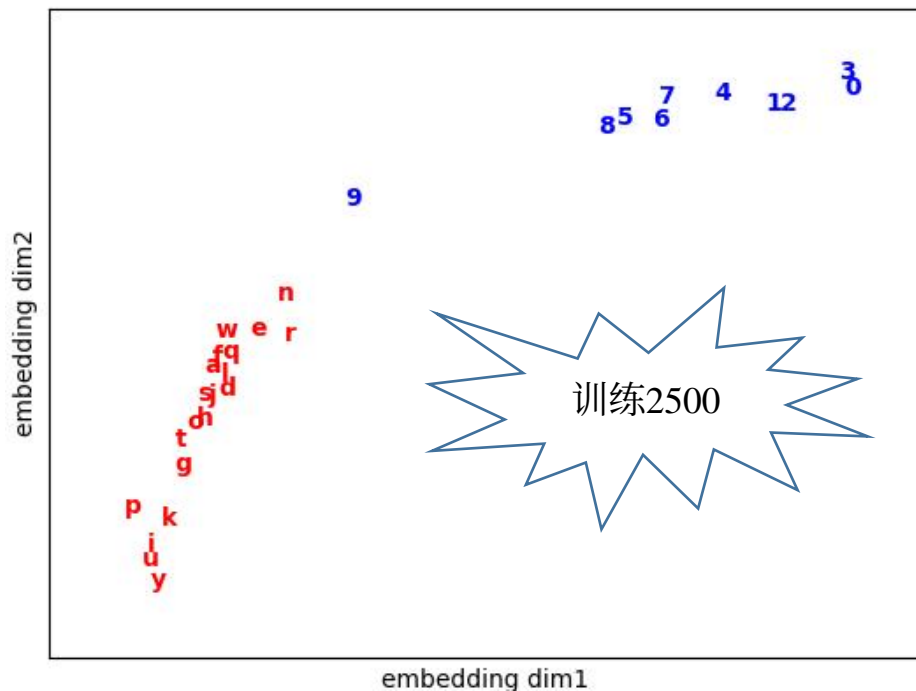
a t g q e h 9 u f

a t g q e h 9 u f

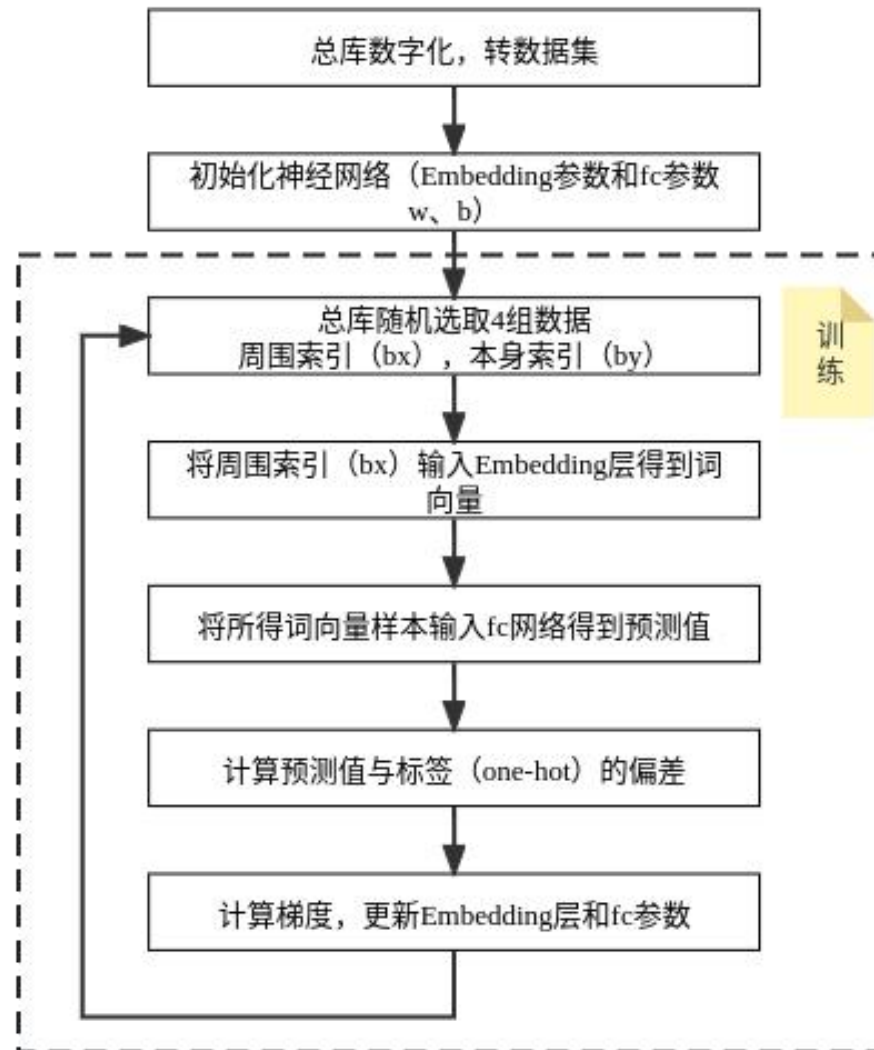
a t g q e h 9 u f

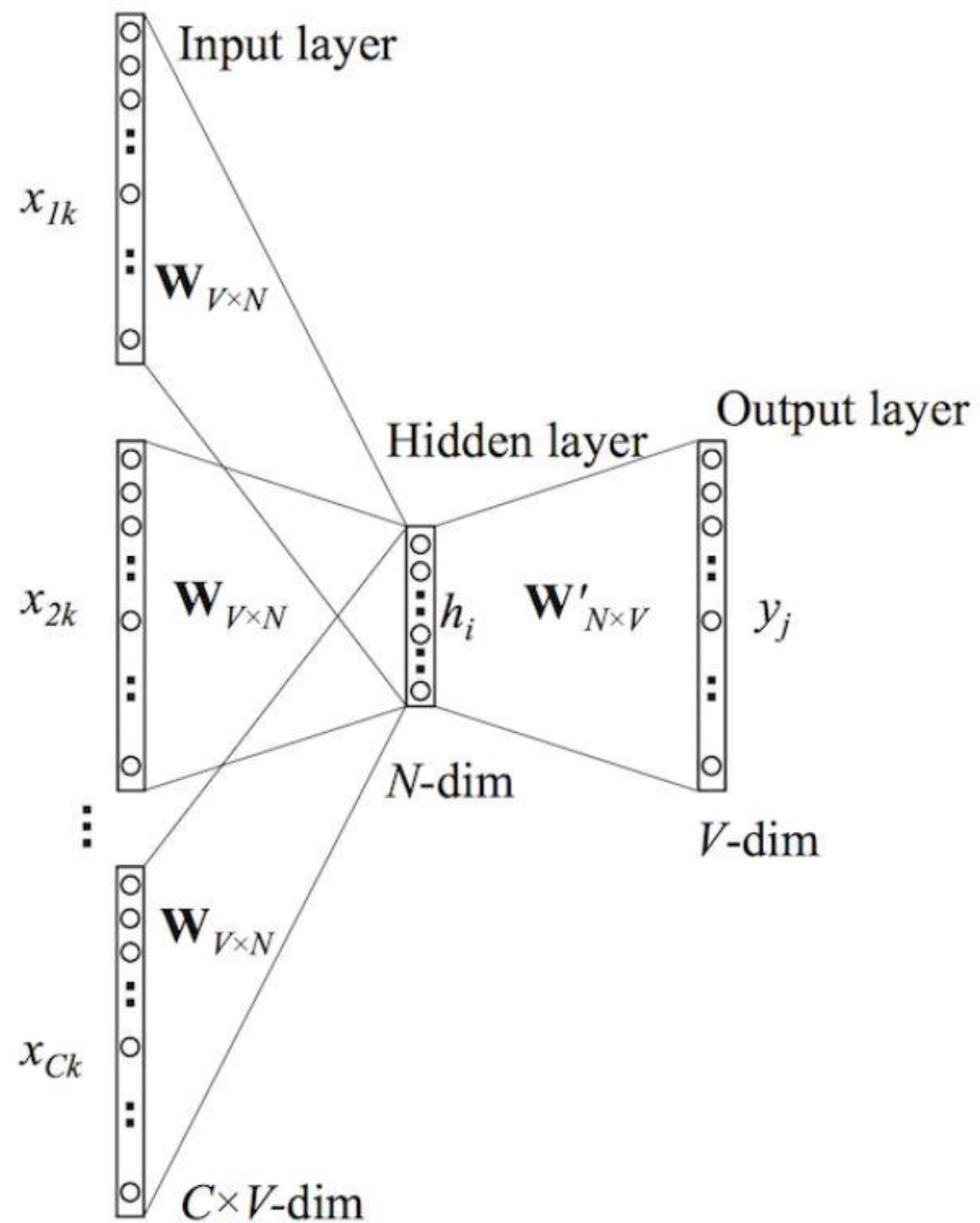
a t g q e h 9 u f

依据深褐色已知元素，预测浅褐色未知元素



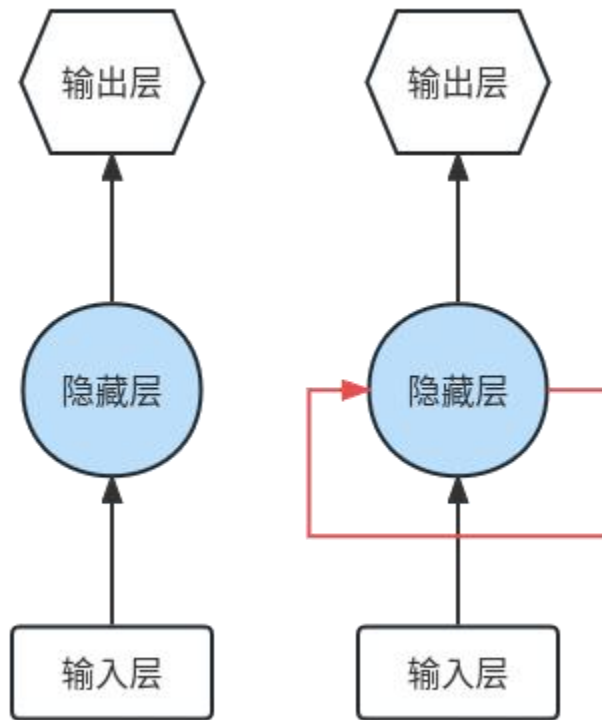
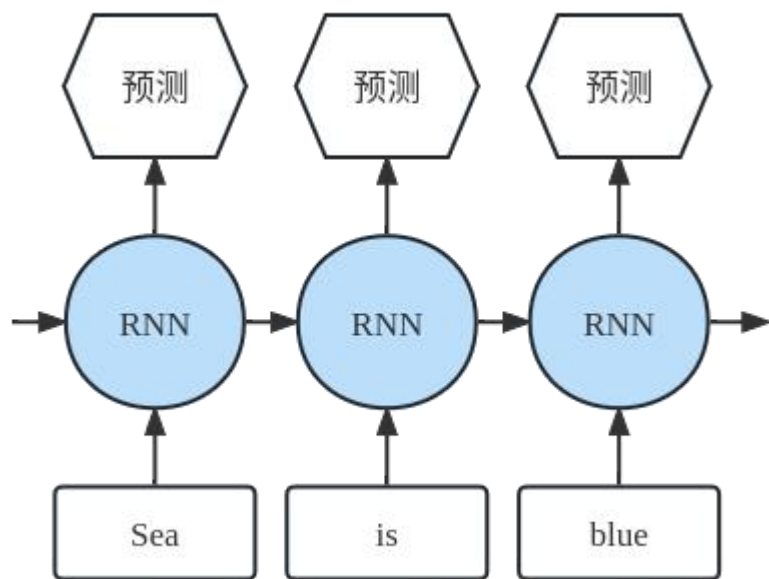
Skin-Gram





递归神经网络 (RNN, Recursion Neural Network)

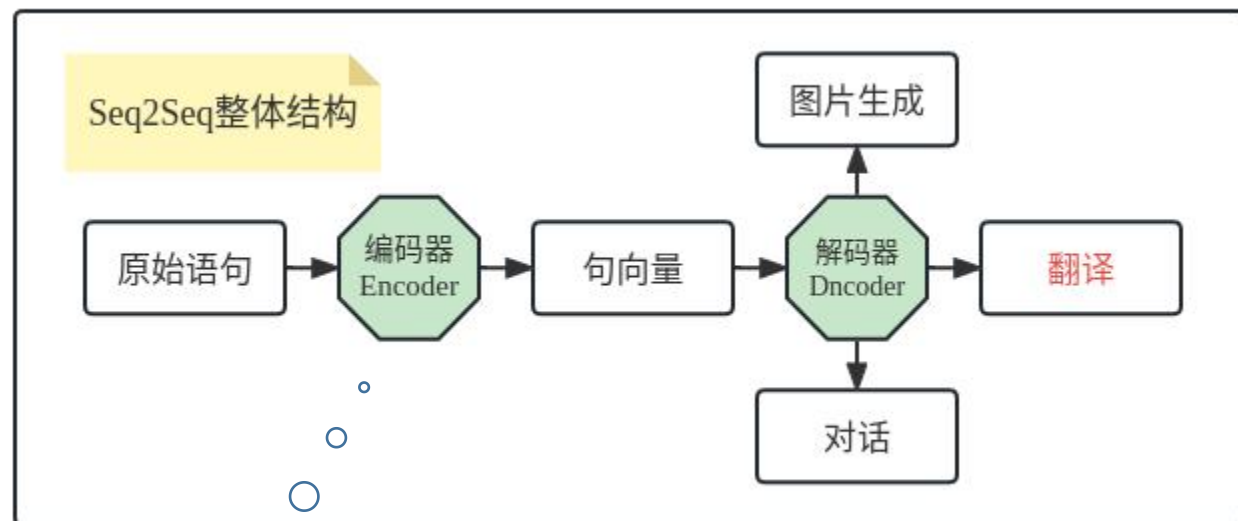
- 使用前馈神经网络时，神经网络会认为我们（ $t+1$ ）时刻输入的内容与 t 时刻输入的内容完全无关。
- NLP分析时，需要合理运用之前的输入。



想象设计了一个基于简单RNN的句子自动补全器，显然第二步的预测值和第一步输入值（‘Sea’）有关。

Seq2Seq 模型

Seq2Seq 是一种循环神经网络的变种，包括编码器 (Encoder) 和解码器 (Decoder) 两部分。



编码器本质是
压缩器

Seq2Seq核心是训练
Encoder和Decoder

例程

目标：训练转化日期格式的模型

31-04-26转化为26/Apr/2031

数据集生成

生成时间戳列表（单位：秒）

列表内容分别转化为两种时间格式
20-05-04 (cn) 04/May/2020 (en)

生成元素字典并排序

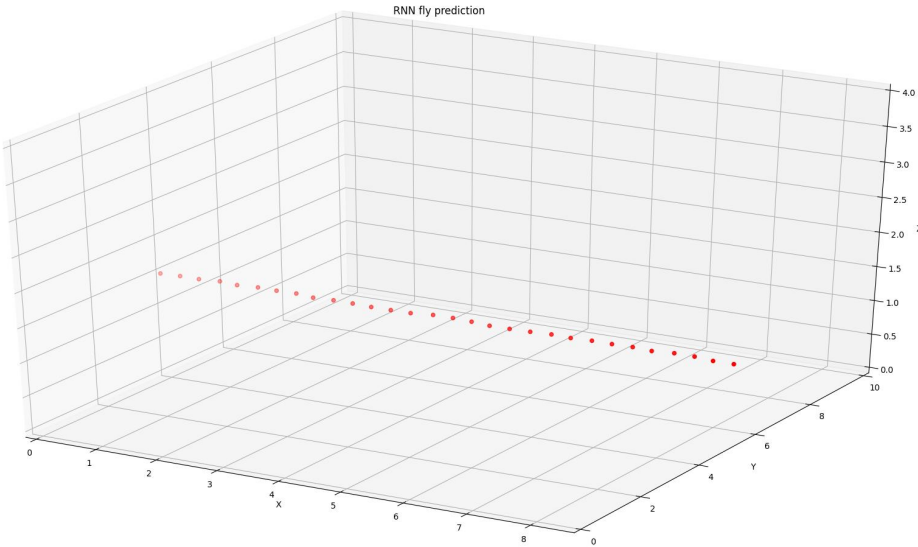
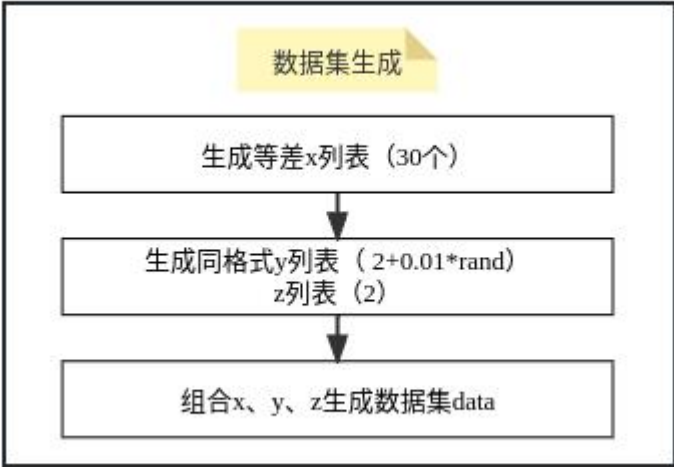
cn格式列表内容转元素索引生成样本x

en格式列表内容转元素索引生成标签y

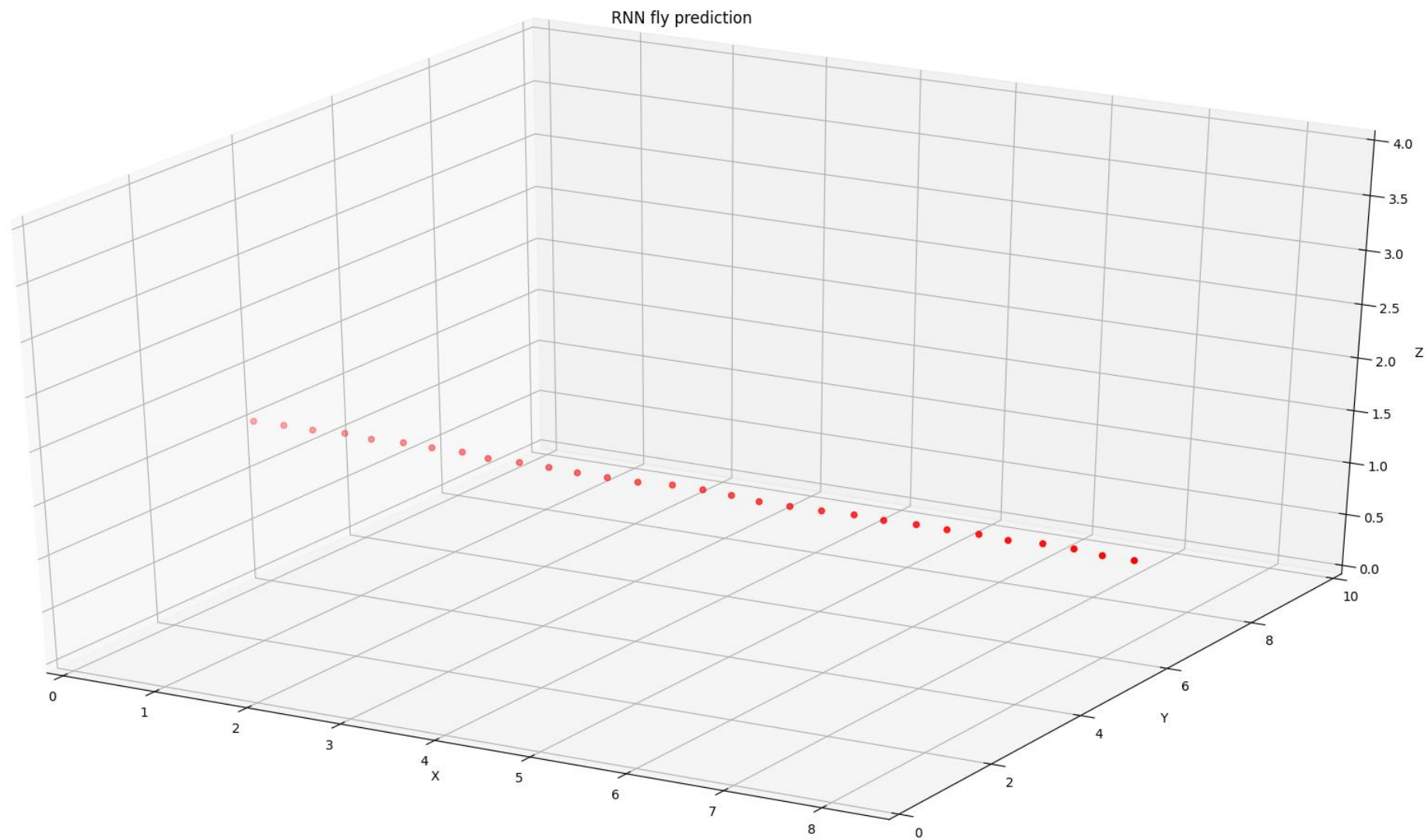
	3	1	-	0	4	-	2	6			
样本	6	4	1	3	7	1	5	9			
	GO	2	6	/	Apr	/	2	0	1	3	EOS
标签	14	5	9	2	15	2	5	3	6	4	13

RNN模型：预测航迹

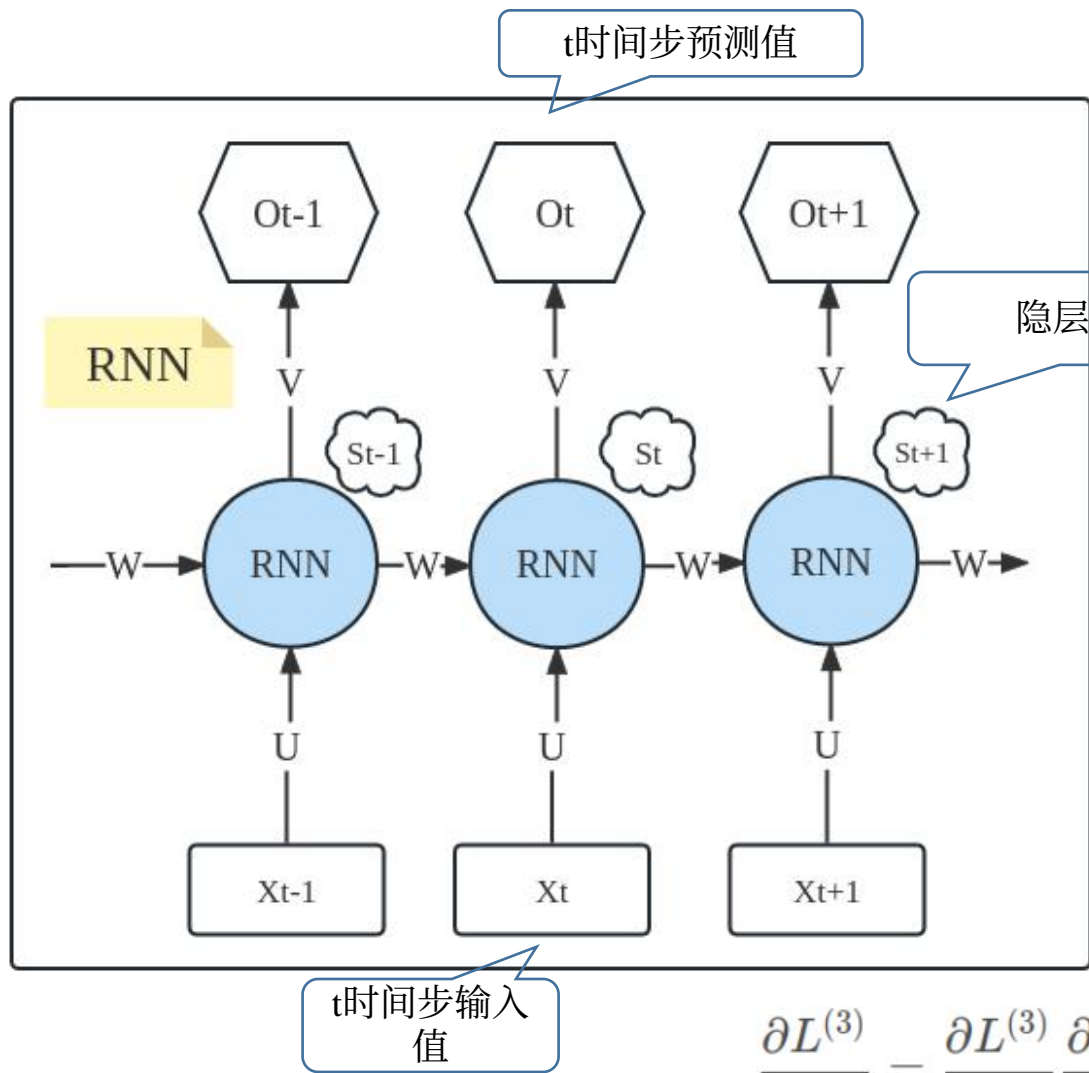
- 模型：依据已知飞机的三维航迹（坐标），预测接下来的航迹数据。



	x	y	z
	0	1	2
0	1.00000	2.00926	2.00000
1	1.31034	2.08682	2.00000
2	1.62069	2.06926	2.00000
3	1.93103	2.00112	2.00000
4	2.24138	2.02131	2.00000
5	2.55172	2.04473	2.00000
6	2.86207	2.05740	2.00000
7	3.17241	2.03513	2.00000
8	3.48276	2.02770	2.00000
9	3.79310	2.09824	2.00000
0	4.10345	2.02728	2.00000
1	4.41379	2.07668	2.00000
2	4.72414	2.07728	2.00000
3	5.03448	2.07636	2.00000
4	5.34483	2.05805	2.00000



RNN原理



$$S_t = \Phi (U x_t + W S_{t-1} + b)$$

$$O_t = V S_t + c$$

$$L = y_t - O_t$$

L: 损失函数
y: 训练集的标签
V、W、U: 权值

$$\frac{\partial L^{(t)}}{\partial V} = \frac{\partial L^{(t)}}{\partial o^{(t)}} \cdot \frac{\partial o^{(t)}}{\partial V}$$

累乘会导致激活函数导数的累乘，进而会导致“梯度消失”和“梯度爆炸”现象的发生。

$$\frac{\partial L^{(3)}}{\partial W} = \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial W} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial W} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(1)}}{\partial W}$$

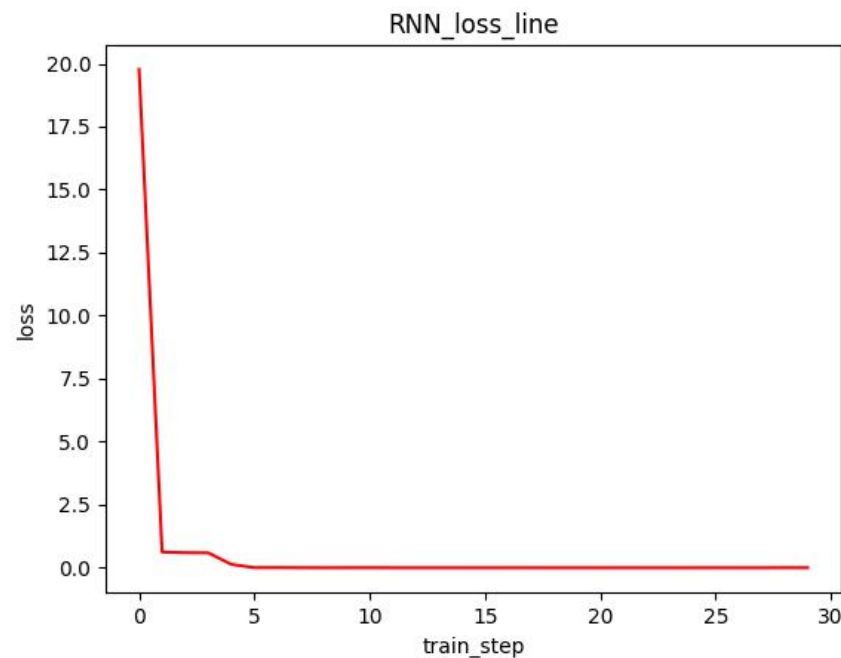
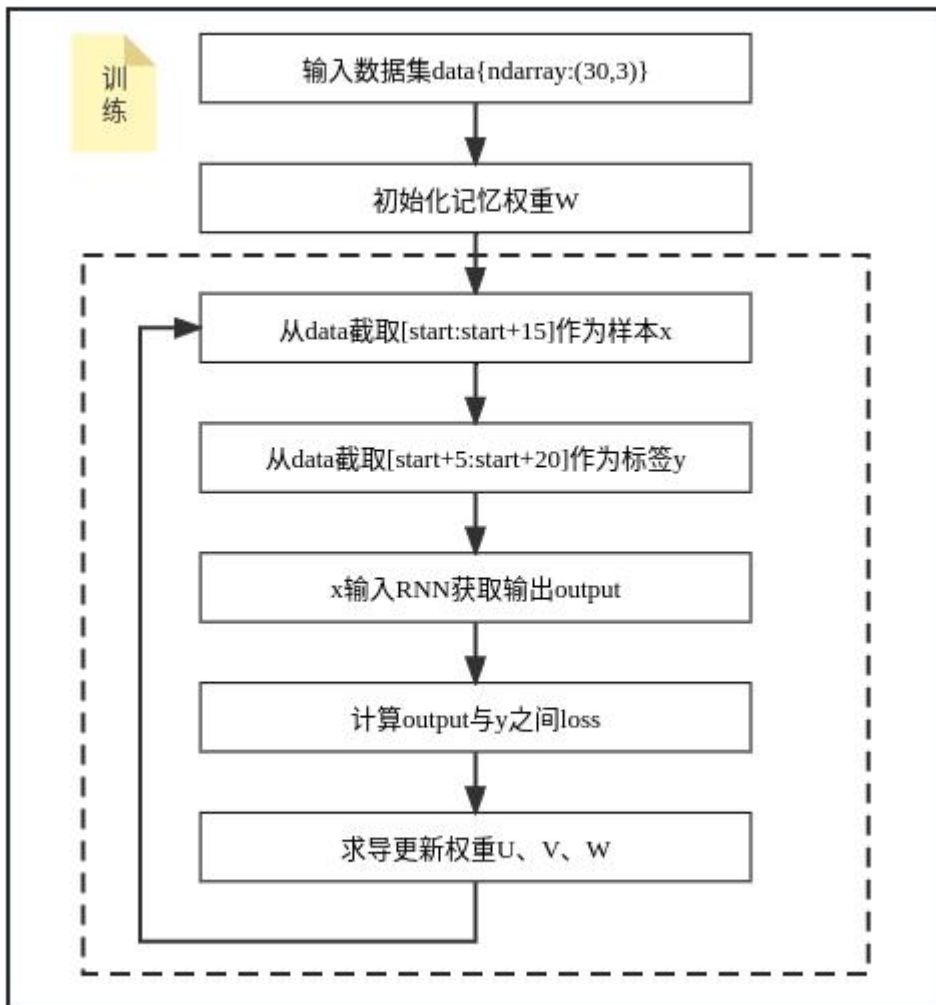
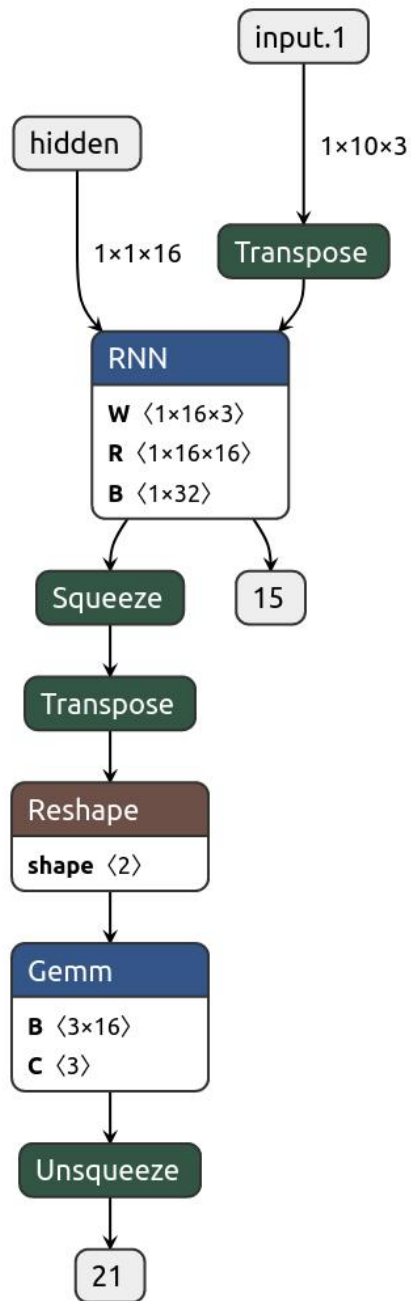
$$\frac{\partial L^{(3)}}{\partial U} = \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial U} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial U} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(1)}}{\partial U}$$

递归神经网络 (RNN, Recursion Neural Network)

$$S_t = \Phi (U x_t + W S_{t-1} + b)$$

$$O_t = V S_t + c$$

$$L = y_t - O_t$$



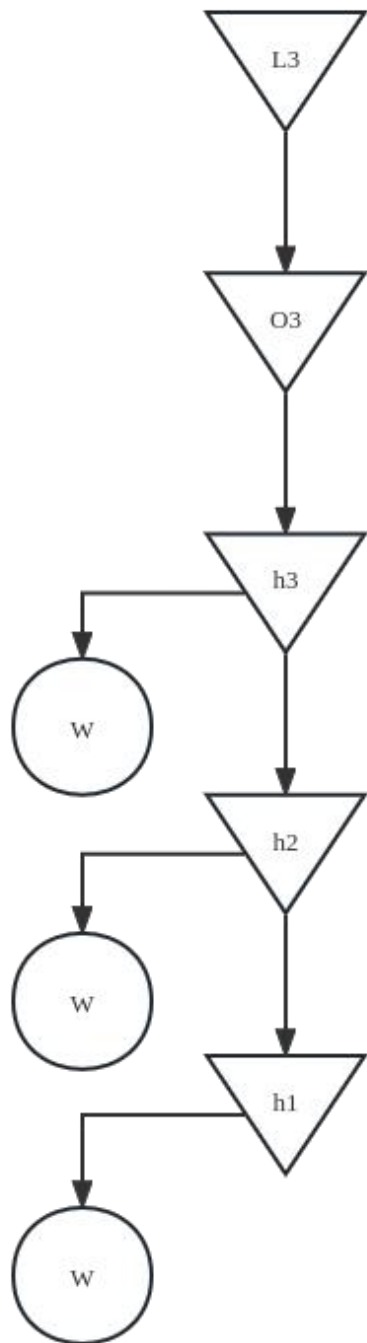
样本

```
tensor([[[[3.7931, 2.0195, 2.0000],
          [4.1034, 2.0125, 2.0000],
          [4.4138, 2.0526, 2.0000],
          [4.7241, 2.0496, 2.0000],
          [5.0345, 2.0179, 2.0000],
          [5.3448, 2.0495, 2.0000],
          [5.6552, 2.0423, 2.0000],
          [5.9655, 2.0452, 2.0000],
          [6.2759, 2.0267, 2.0000],
          [6.5862, 2.0820, 2.0000]]]])
```

输出

```
tensor([[[[5.3063, 2.0607, 1.9949],
          [5.6160, 2.0377, 1.9953],
          [5.9566, 2.0423, 1.9962],
          [6.2701, 2.0509, 1.9964],
          [6.5869, 2.0598, 1.9968],
          [6.8966, 2.0680, 1.9974],
          [7.2097, 2.0732, 1.9976],
          [7.5217, 2.0749, 1.9975],
          [7.8394, 2.0727, 1.9971],
          [8.1443, 2.0679, 1.9967]]]])
```

14	5.34483	2.04946	2.00000
15	5.65517	2.04226	2.00000
16	5.96552	2.04522	2.00000
17	6.27586	2.02667	2.00000
18	6.58621	2.08199	2.00000
19	6.89655	2.06064	2.00000
20	7.20690	2.09343	2.00000
21	7.51724	2.07748	2.00000
22	7.82759	2.09709	2.00000
23	8.13793	2.04459	2.00000
24	8.44828	2.09234	2.00000



$$H_t = \Phi (U x_t + W H_{t-1} + b)$$

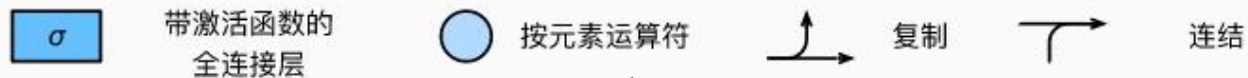
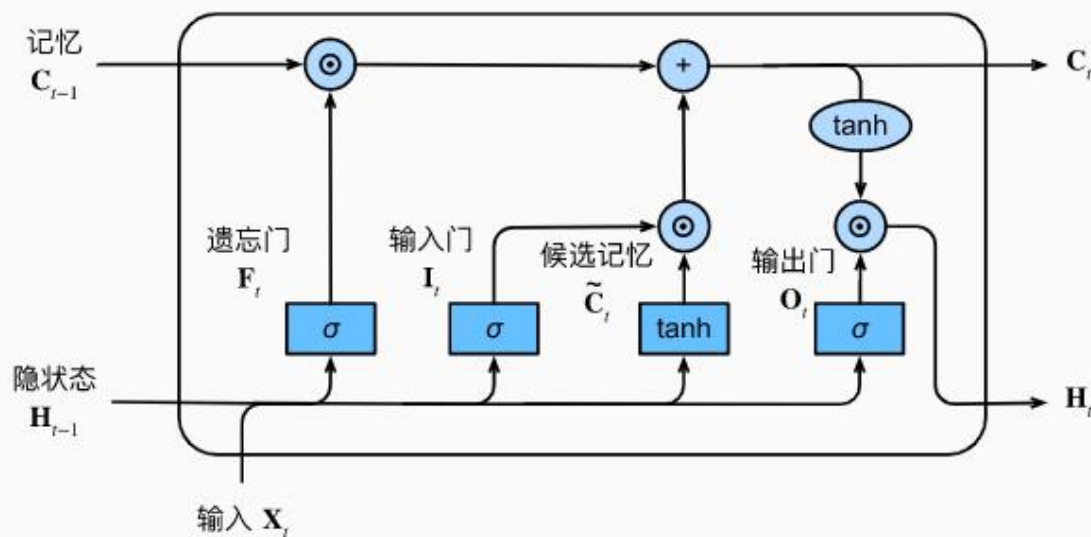
$$O_t = V H_t + c$$

$$L = y_t - O_t$$

$$\frac{\partial L^{(3)}}{\partial W} = \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial W} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial W} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(1)}}{\partial W}$$

$$\frac{\partial L^{(3)}}{\partial U} = \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial U} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial U} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(1)}}{\partial U}$$

LSTM



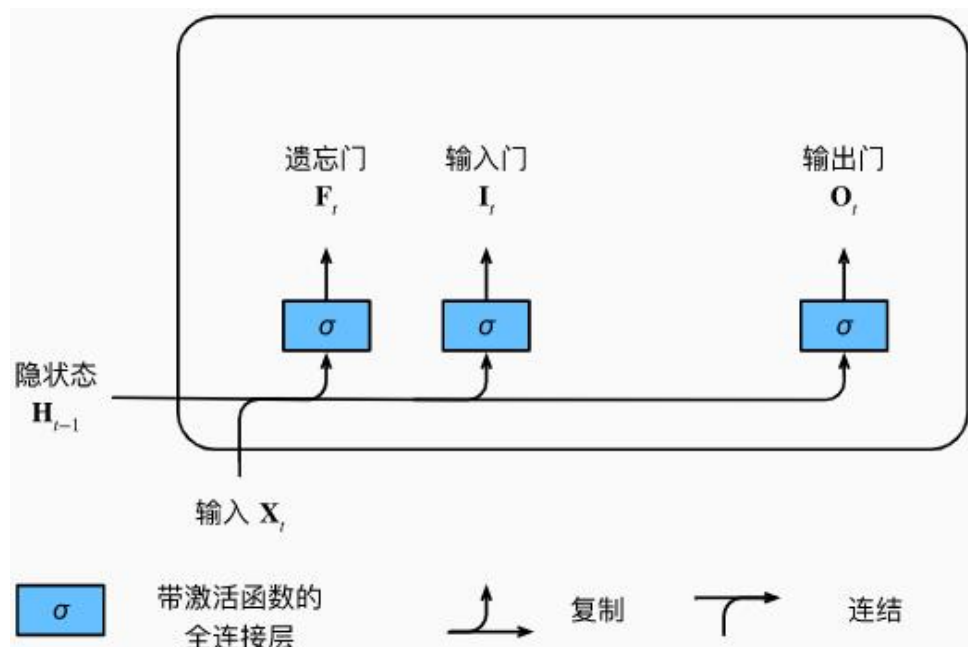
相对于RNN, LSTM有各种“门”处理信息

记忆 (C_t) : 每个LSTM单元保留一个不输出的内部状态, 也称为细胞状态。是对于所有输入信息的“概括总结”

隐藏状态 H_t : 直接与输出状态相关。

- 通过训练让“门”学习保存哪些信息? 丢弃哪些信息?
- 保存的信息哪些输出? 哪些不输出?

输入门 I_t 、忘记门 F_t 和输出门 O_t



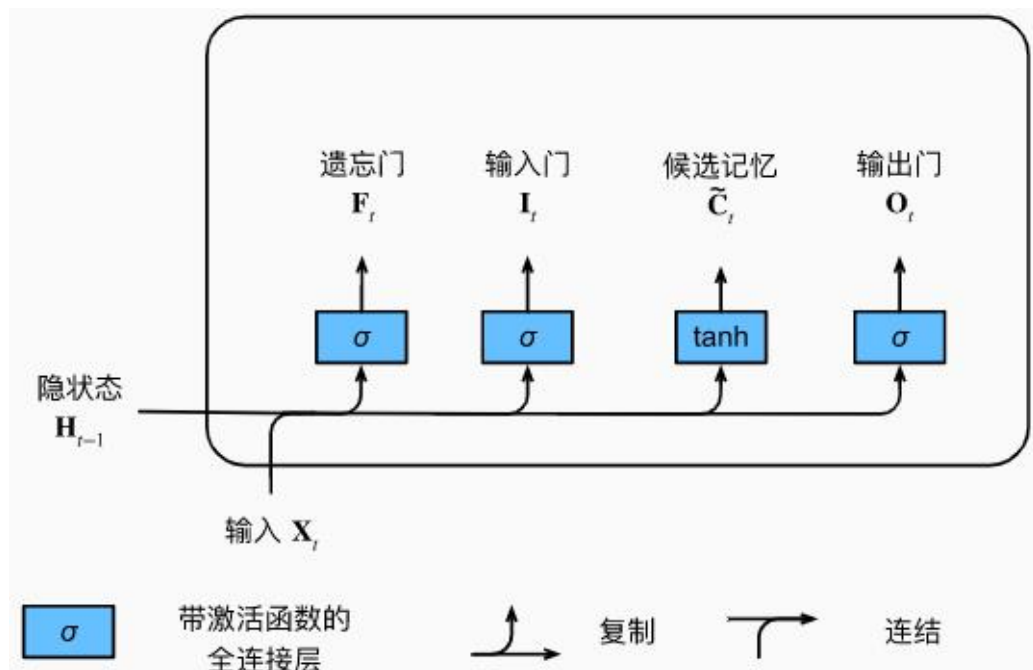
当前时间步的输入 X_t 和前一个时间步的隐状态 H_{t-1} 作为数据送入LSTM，由三个具有sigmoid激活函数的全连接层处理，以计算输入门、忘记门和输出门的值。这三个门的值都在(0,1)的范围内。

- 忘记门:根据新的输入和上一时刻的输出决定遗忘掉之前的哪些记忆。
- 输入门: 决定让多少新的信息加入到 cell 状态 中来。
- 输出门: 计算当前时刻的输出值的神经层。

$$\begin{aligned} I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i), \\ F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f), \\ O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o), \end{aligned}$$

建立I、F、O与隐
状态 H_{t-1} &样本 X_t
之间的关系。

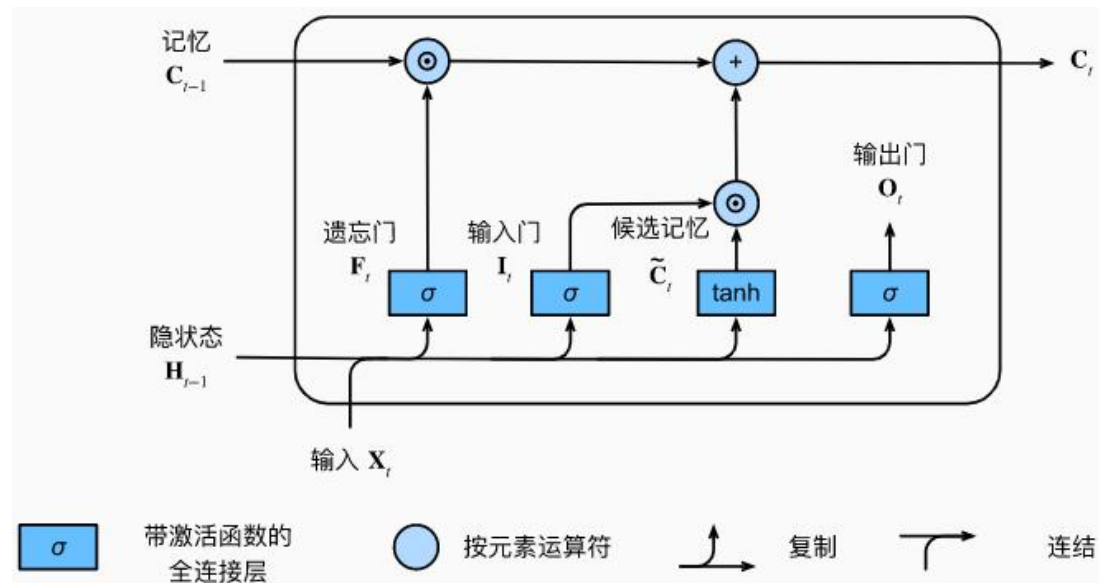
候选记忆元



$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c)$$

- 将输入的 x_t 和 h_{t-1} 整合，然后通过一个tanh神经网络层来创建一个新的状态候选向量 $\hat{\mathbf{C}}_t$

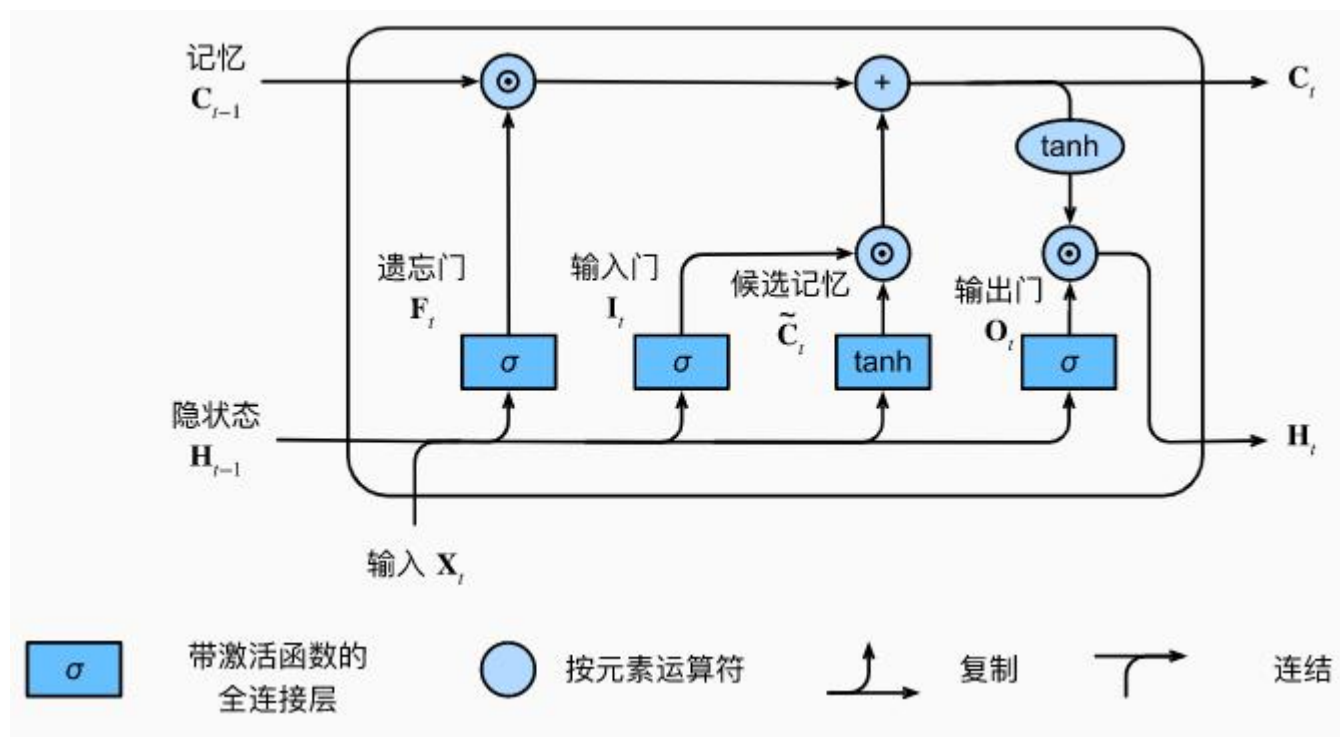
记忆元



$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t.$$

- 输入门 I_t 控制采用多少来自 $\hat{\mathbf{C}}_t$ 的新数据
- 遗忘门 F_t 控制保留多少过去的记忆元 $\mathbf{C}_{t-1}$ 的内容。

隐状态

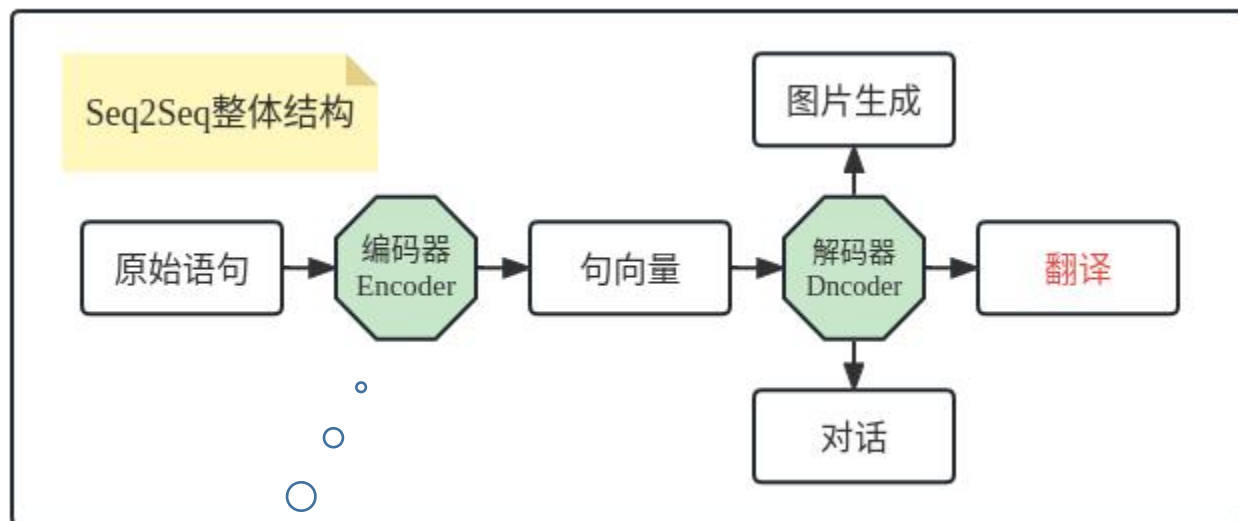


- 隐状态会传递到输出层
- 记忆元完全属于内部信息

$$H_t = O_t \odot \tanh(C_t)$$

Seq2Seq 模型

Seq2Seq 是一种循环神经网络的变种，包括编码器 (Encoder) 和解码器 (Decoder) 两部分。



编码器本质是
压缩器

Seq2Seq核心是训练
Encoder和Decoder

例程

目标：训练转化日期格式的模型

31-04-26转化为26/Apr/2031

数据集生成

生成时间戳列表（单位：秒）

列表内容分别转化为两种时间格式
20-05-04 (cn) 04/May/2020 (en)

生成元素字典并排序

cn格式列表内容转元素索引生成样本x

en格式列表内容转元素索引生成标签y

	1	2	3	4	5	6	7	8	9	10	11
	3	1	-	0	4	-	2	6			
样本	6	4	1	3	7	1	5	9			
	GO	2	6	/	Apr	/	2	0	1	3	EOS
标签	14	5	9	2	15	2	5	3	6	4	13

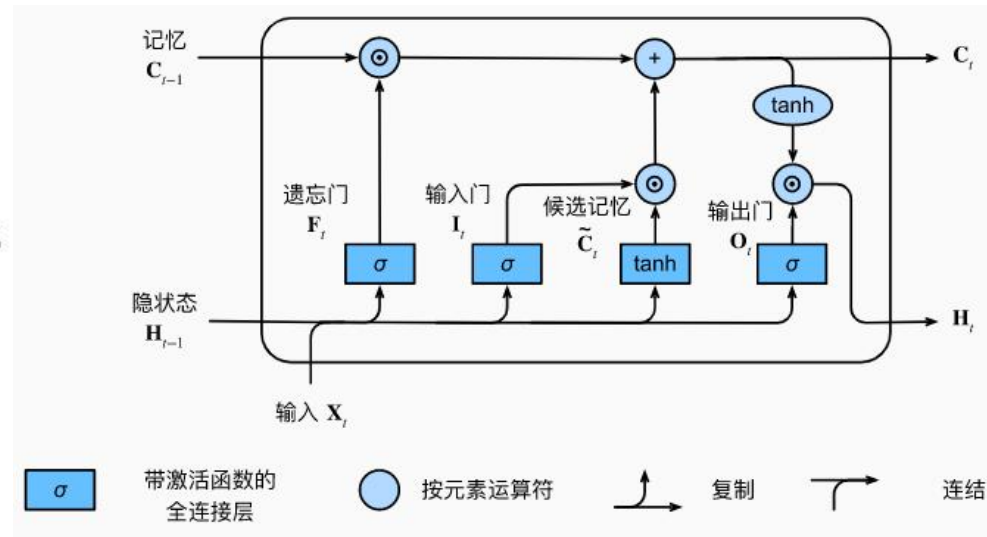
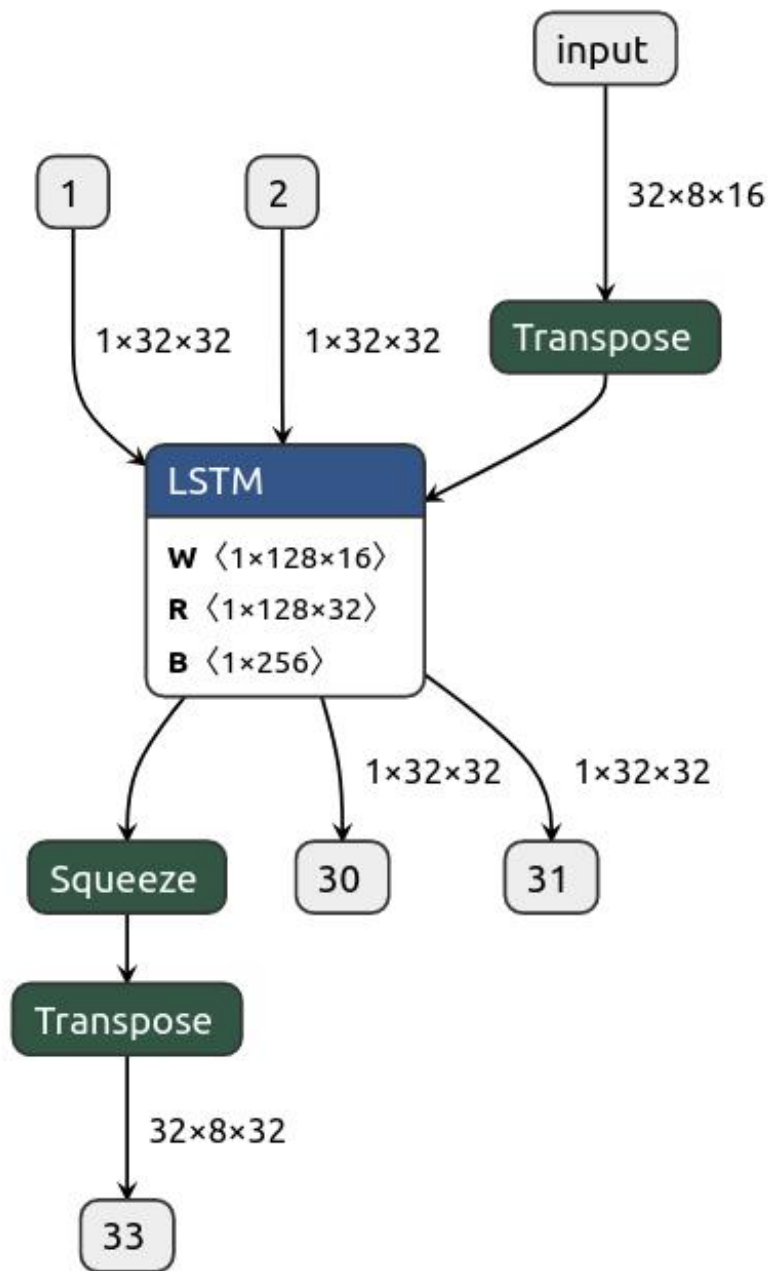
压缩器encoder

作用：将原始的词向量按顺序组装起来变成句向量。

input{tensor:(32,8)}: 32样本, 每个样本8个one-hot编码 (27元素)

- 1&30: 初始隐藏状态
- 2&31: 初始细胞状态
- 33: 输出

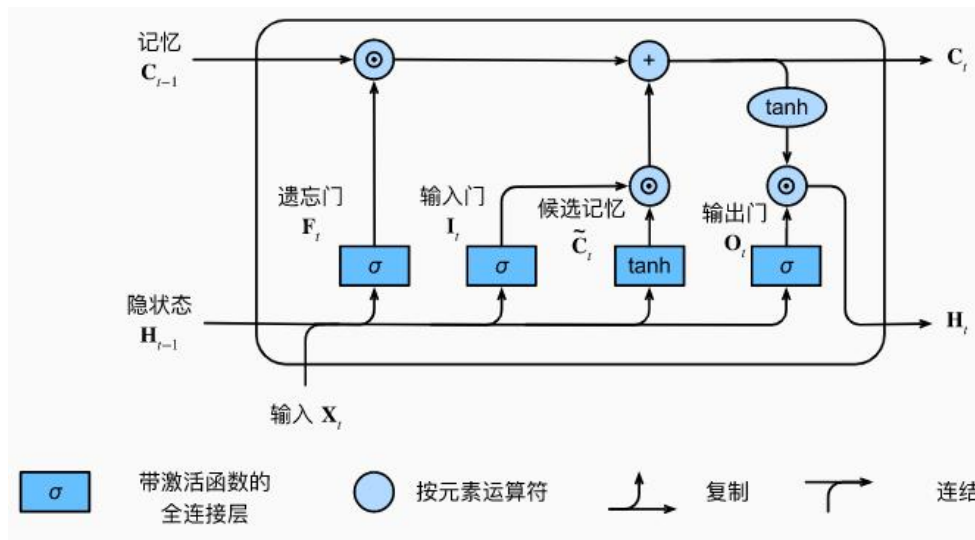
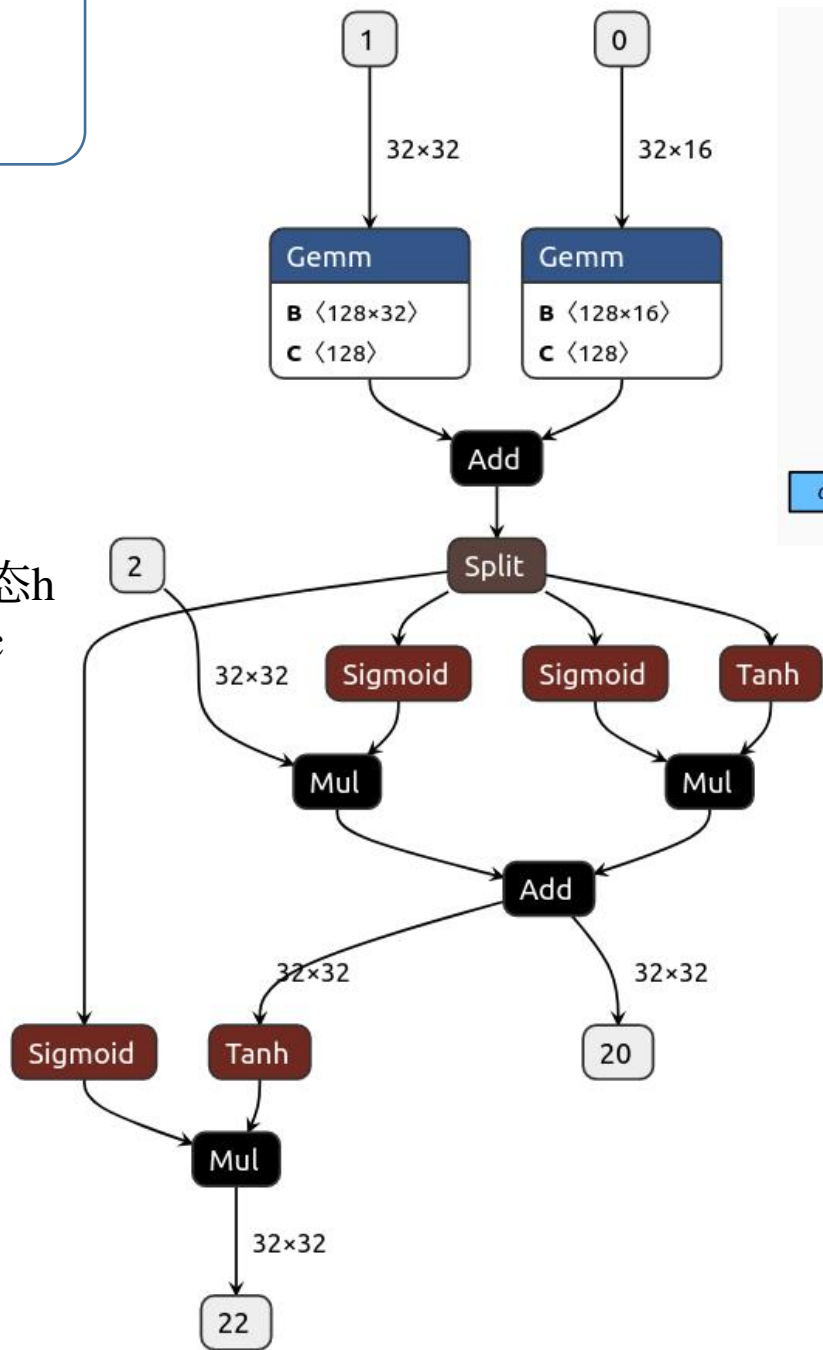
- 86-10-14



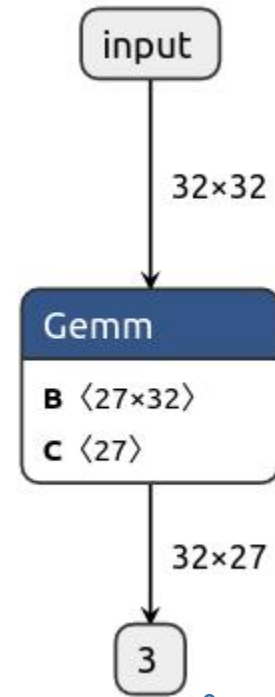
将样本每个字符作为 X_i 依次输入压缩器LSTM，获取的最后一个记忆和最后一个隐状态作为初始记忆和初始隐状态输入解压器

解压器decoder

- 0: 词向量
- 1&22: 隐状态h
- 2&20: 记忆c



Split-->Sigmoid-->Mul (右) 遗忘门
 Split-->Sigmoid-->Mul (左) 输入门
 Split-->Tanh-->Mul 候选记忆
 Split-->Sigmoid-->Mul-->22 输出



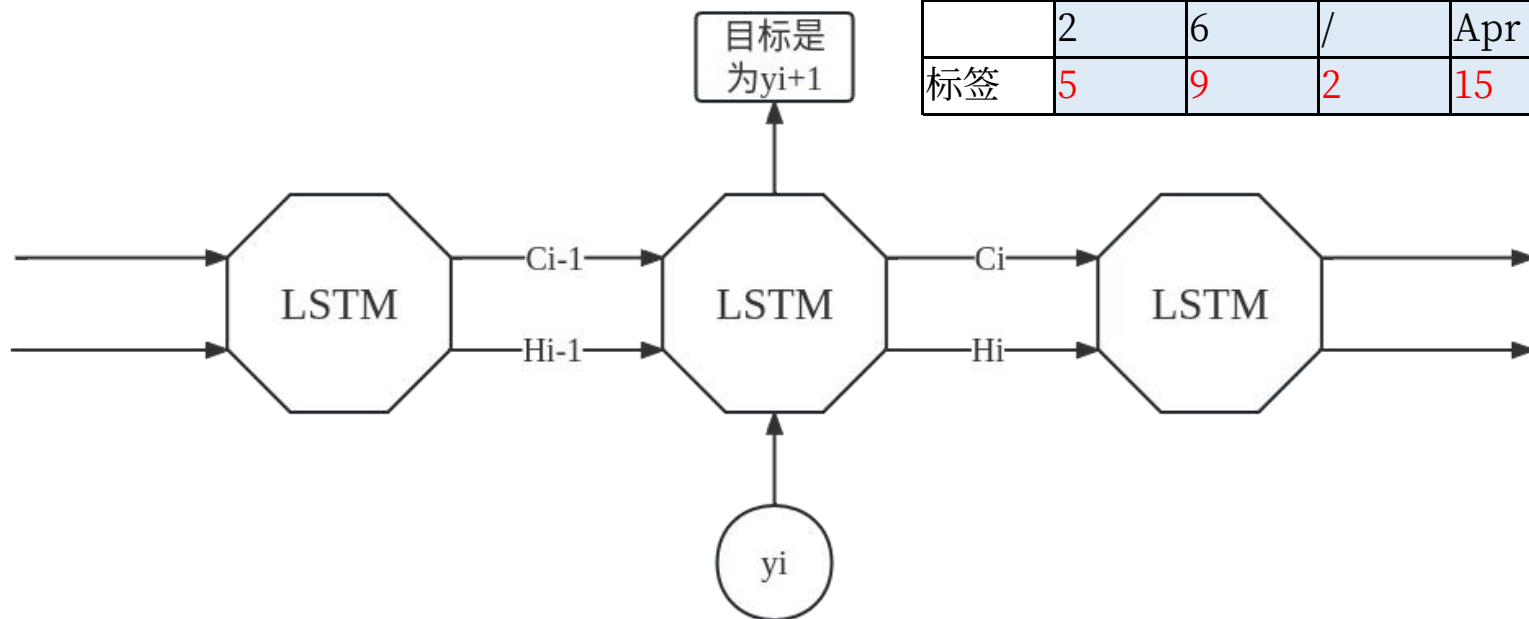
将隐状态转为
概率分布

解压器decoder

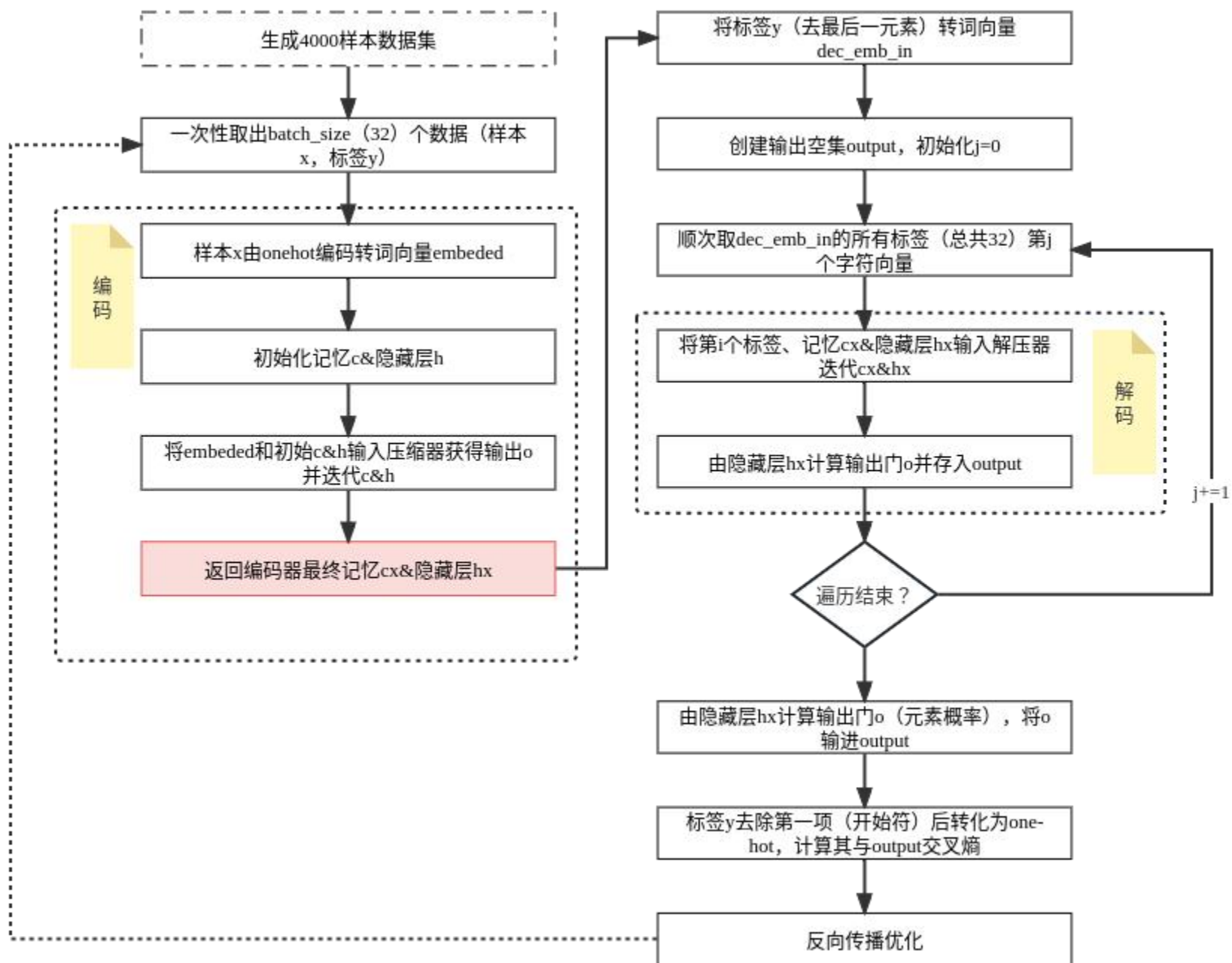
- 解压器输入为样本第 i 个字符，输出为样本 $i+1$ 个字符在元素集合的概率分布

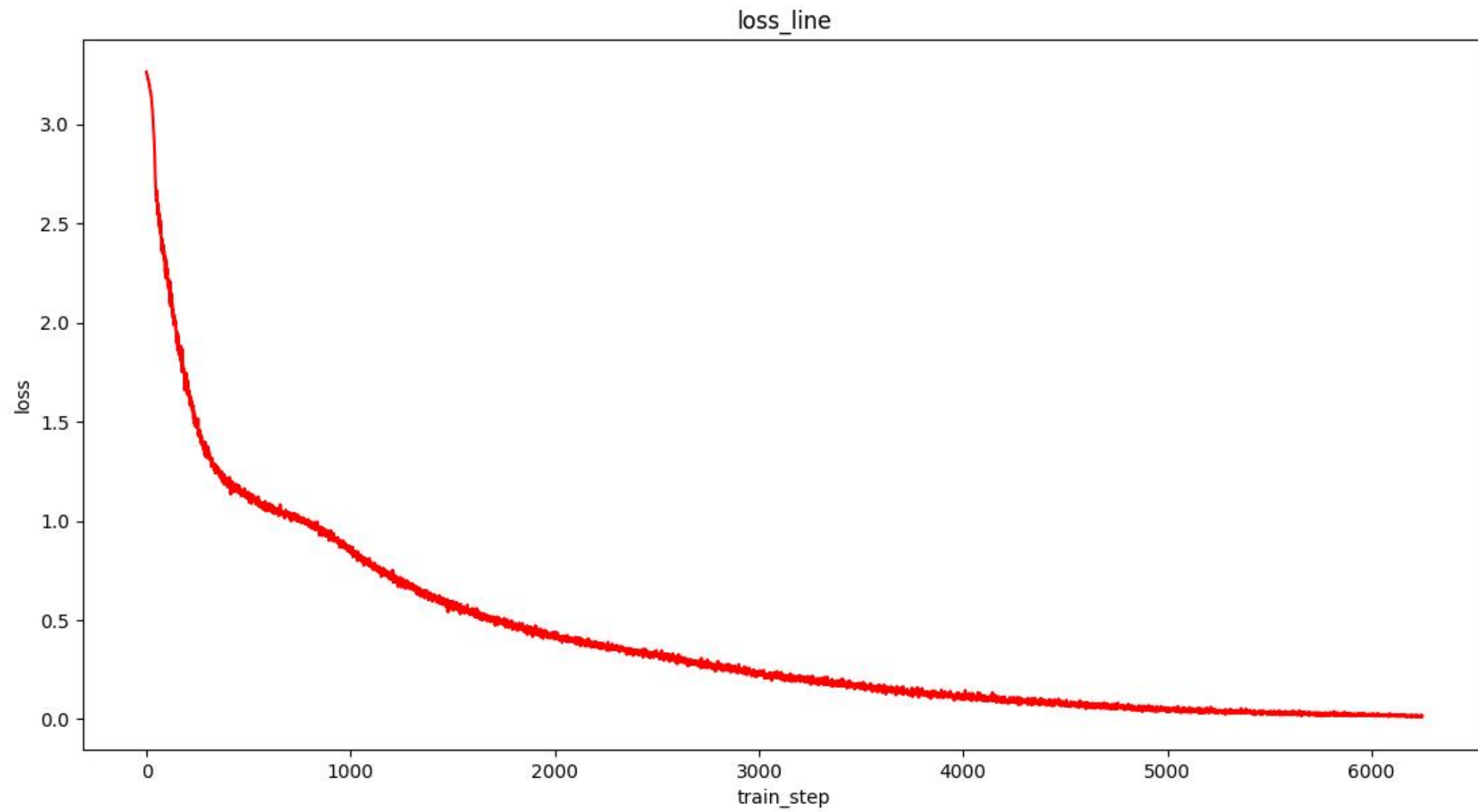
	1	2	3	4	5	6	7	8	9	10	11
	3	1	-	0	4	-	2	6			
样本	6	4	1	3	7	1	5	9			
	GO	2	6	/	Apr	/	2	0	1	3	EOS
标签	14	5	9	2	15	2	5	3	6	4	13

	1	2	3	4	5	6	7	8	9	10	11
	GO	2	6	/	Apr	/	2	0	1	3	EOS
样本	14	5	9	2	15	2	5	3	6	4	13
	2	6	/	Apr	/	2	0	1	3	EOS	
标签	5	9	2	15	2	5	3	6	4	13	



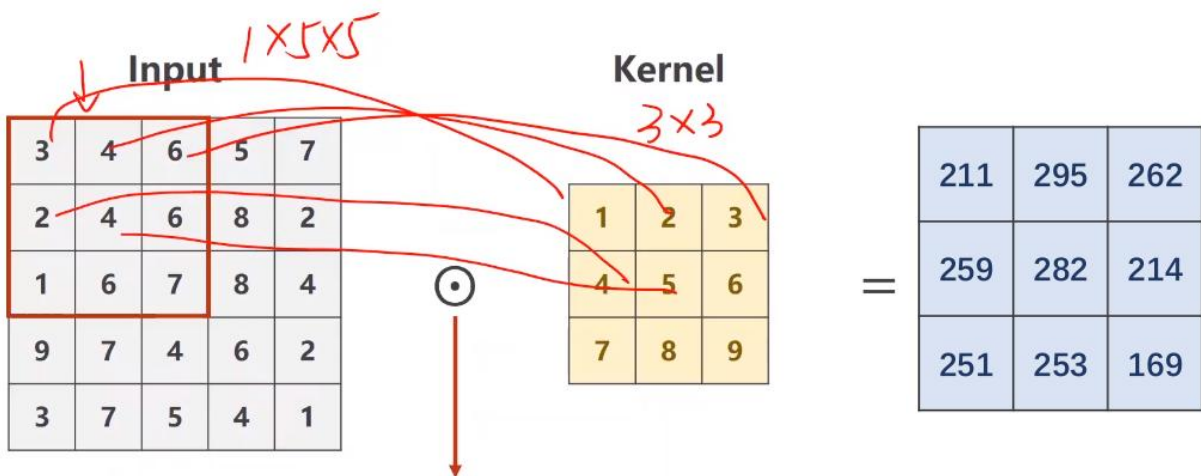
- 解压器训练时输入为标签
- 使用时输入为起始符GO和上一步解压器的输出





2023.03.15

卷积神经网络



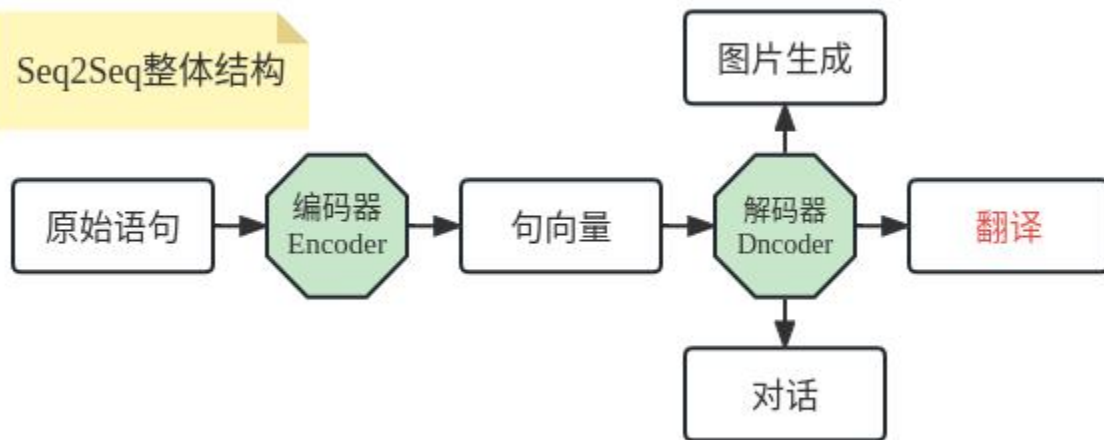
$$3 \cdot 1 + 4 \cdot 2 + 6 \cdot 3 + 2 \cdot 4 + 4 \cdot 5 + 6 \cdot 6 + 1 \cdot 7 + 6 \cdot 8 + 7 \cdot 9 =$$

$$3 + 8 + 18 + 8 + 20 + 36 + 7 + 48 + 63 = 211$$

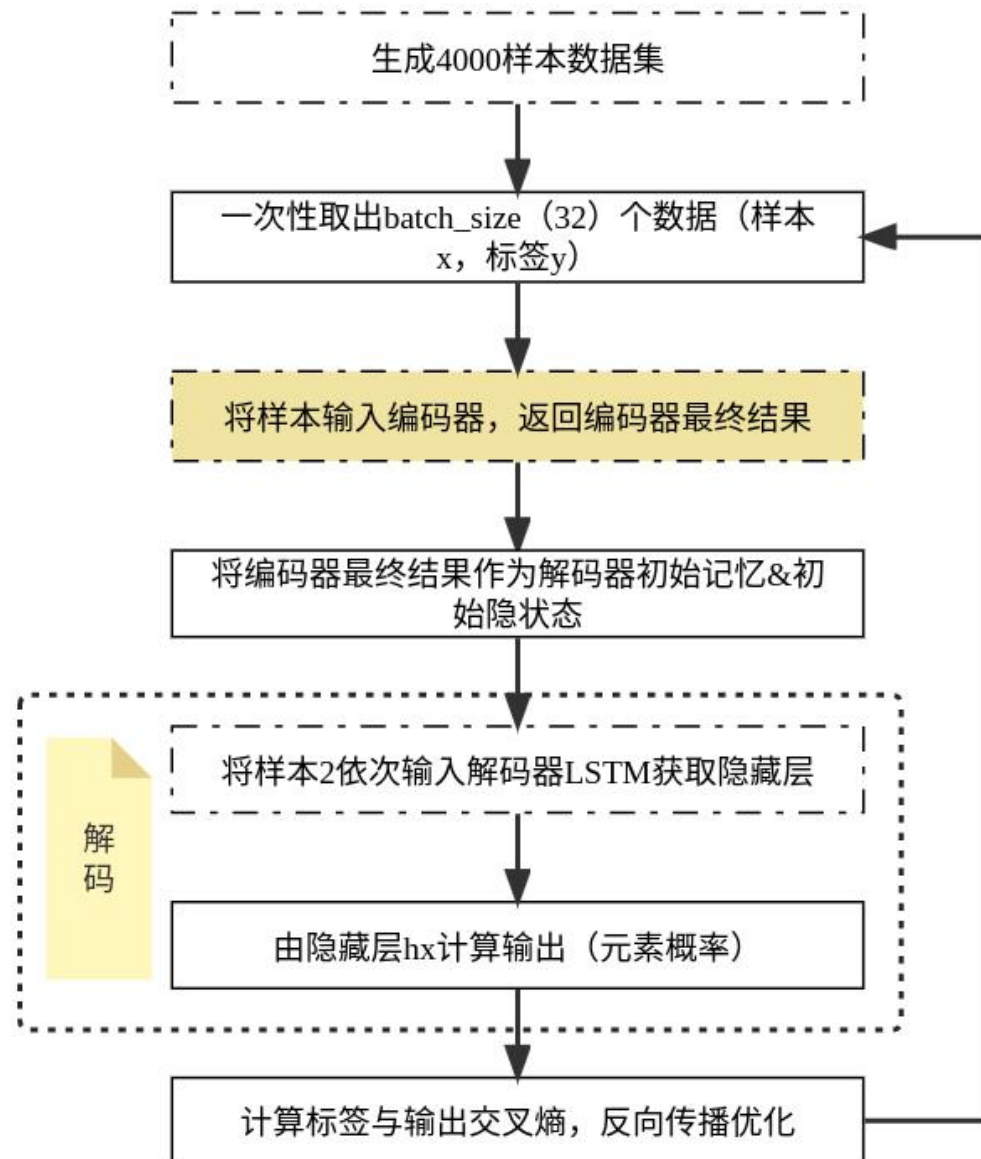
$$4 \cdot 1 + 6 \cdot 2 + 5 \cdot 3 + 4 \cdot 4 + 6 \cdot 5 + 8 \cdot 6 + 6 \cdot 7 + 7 \cdot 8 + 8 \cdot 9 =$$

$$4 + 12 + 15 + 16 + 30 + 48 + 42 + 56 + 72 = 295$$

Seq2Seq整体结构



- 本代码与上一代代码功能相同，结构相同
- 编码器结果仍然作为初始记忆和初始隐状态输入解码器
- 解码器没有变化
- 不同之处是编码器主要内容为CNN



CNN压缩器encoder

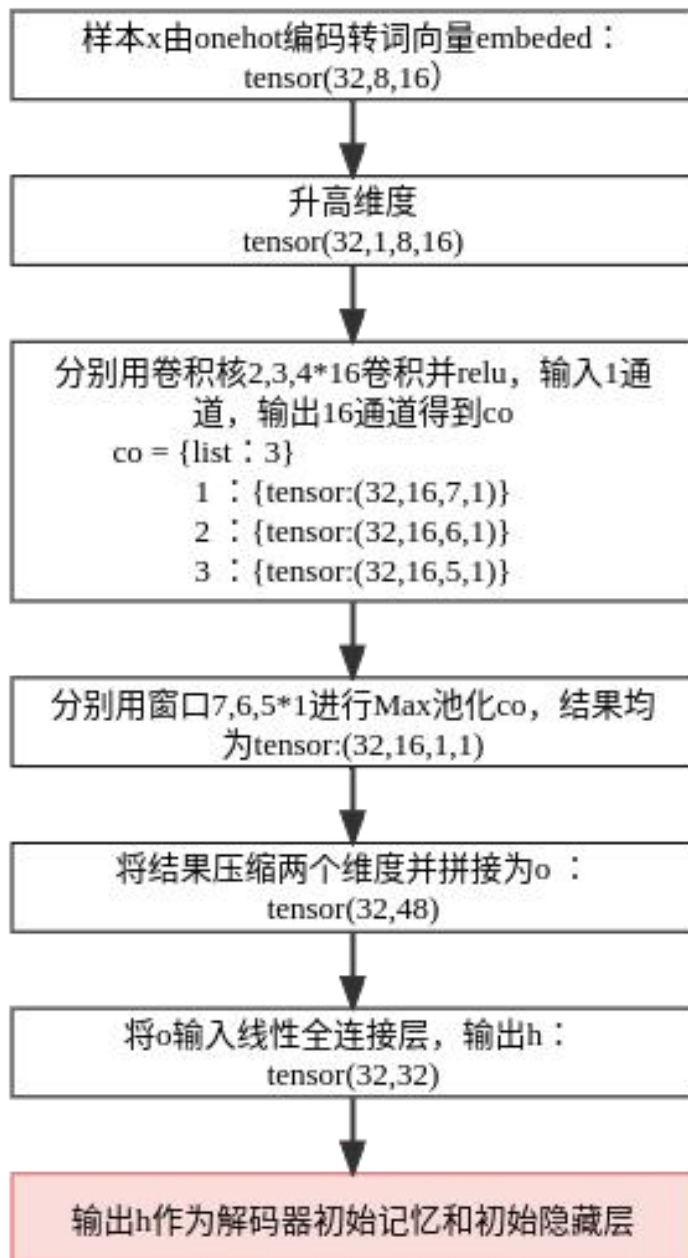
- 86-10-14

{tensor:(32,1,8,16)}

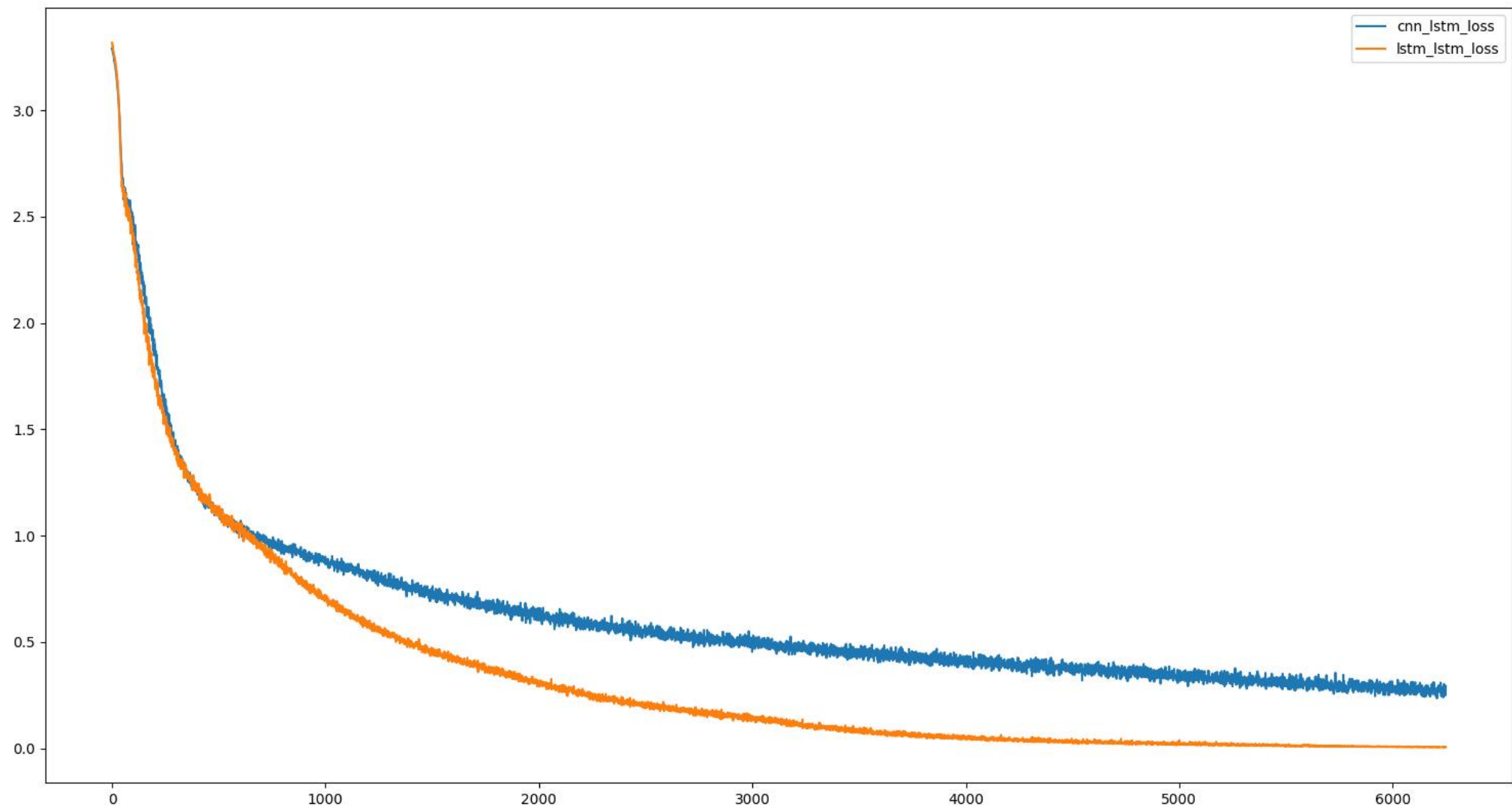
	字符 1		字符8
16维 词向 量		...	
	...	...	...
		...	

卷积器: nn.Conv2d(1,16,(3,16))
输入通道1,输出通道16,卷积核大小
为 3×16

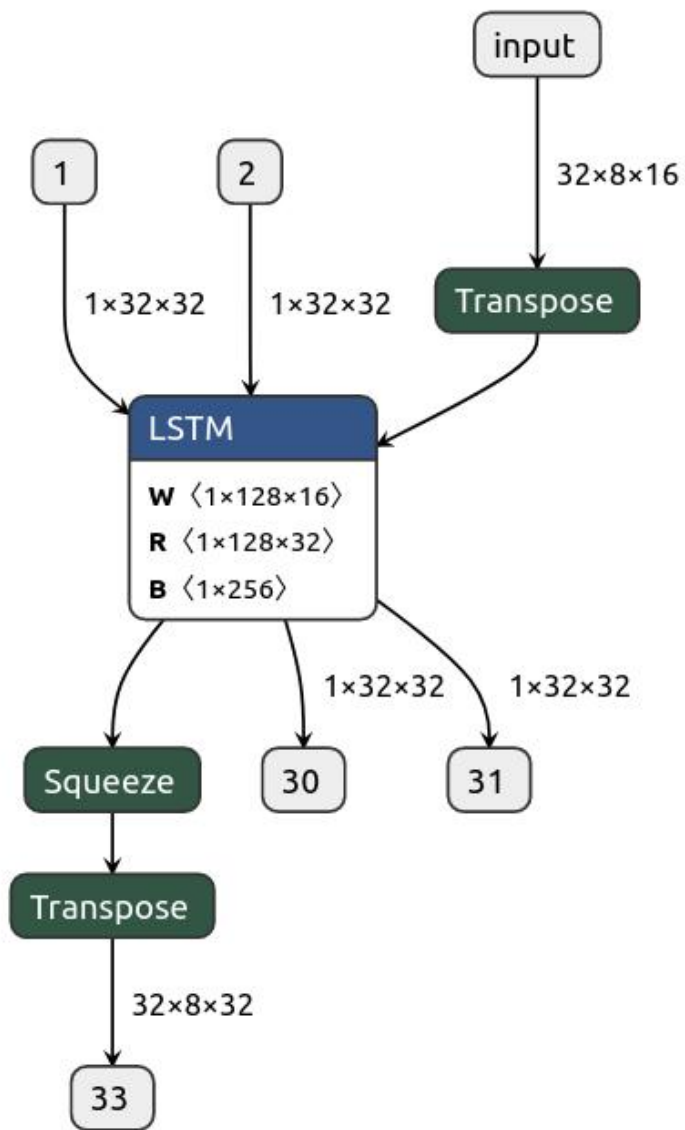
{tensor:(32,16,6,1)}



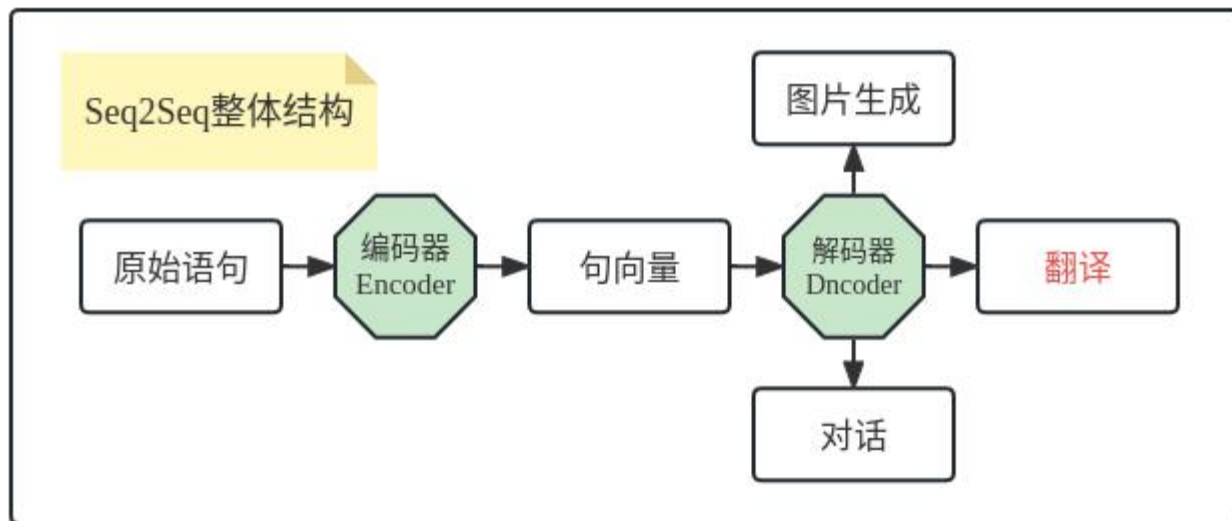
- CNN局限性: 要求有个句子最长的限制, 句子如果超过这个长度, 一般只能截断
- 根据CNN编码器与LSTM编码器的对比, 可以看出相同训练步, CNN编码器loss略大



attention解码器
encoder



- 本代码与上一代代码功能相同，主体结构相同
- LSTM编码器每步输出门（33）也保存并输出到解码器
- 编码器内部结构相同，编码器最终隐藏层和记忆仍然作为初始记忆和初始隐状态输入解码器
- 解码器引入attention



- 中间状态（句向量）来自于输入网络最后的隐藏层，一般来说它是一个大小固定的向量，能储存的信息就是有限的（当前瓶颈）

attention解码器
encoder

